AI-Native Engineering Curriculum

6-month day-by-day plan for a senior MERN developer moving toward Level 2 strength and Level 3 foundations

Target	Outcome
By Month 3	Foundation correct: LLM APIs, structured outputs, tool calling, RAG, agents from scratch, LangGraph basics.
By Month 6	Strong Level 2: production RAG, agents, MCP server, evals, observability, security, deployment.
After Month 6	Midway into Level 3: agent runtime, tool gateway, policy layer, queue-based execution, platform architecture.
Daily commitment	2 hours minimum, 3 hours recommended, 4 hours when deep work is possible.

Non-negotiable rule: every week must produce a working artifact. Watching courses without shipping code does not count.

Prepared for: Senior full-stack MERN developer

Time horizon: 182 days / 26 weeks / 6 months

1. What this curriculum is designed to do

Your goal is not to become a machine learning researcher. Your goal is to become an AI-native software engineer who can build, evaluate, secure, and operate AI systems. That means staying strong in product engineering while adding LLM systems, RAG, agents, MCP, evals, observability, and infrastructure thinking.

The plan assumes you are already strong in MERN: APIs, backend logic, frontend state, database design, authentication, deployment, and debugging. Those skills remain valuable. The upgrade is learning how to add probabilistic model behavior, tool execution, retrieval, and long-running workflows without building fragile demos.

The target is realistic: in 6 months you can become strong at Level 2 and establish Level 3 foundations. Reaching serious Level 3 usually takes continued work after this plan, especially in distributed systems, security, runtime design, and model infrastructure.

The three levels

Level	What it means	Risk if you stop there
Level 1 - AI tool user	Uses no-code tools, prompt templates, workflow builders, and AI coding tools.	Crowded and easy to copy. Useful, but not a durable senior engineering moat.
Level 2 - AI application engineer	Builds RAG systems, agents, tool-calling apps, copilots, AI backends, evals, and production workflows.	Strong market relevance, but must keep improving infra/security depth.
Level 3 - AI systems / infra engineer	Builds agent runtimes, MCP servers, tool gateways, eval platforms, model routing, observability, secure execution, and developer platforms.	Higher bar. Requires architecture maturity, backend depth, security thinking, and operational discipline.

What you will be able to build

- A production-shaped AI backend using Node/TypeScript and Python services.
- A repo-aware RAG assistant that answers questions with grounded citations.
- An agent runtime from scratch with tools, memory, state, stop conditions, approvals, and traces.
- A LangGraph workflow with persistence, branching, human-in-the-loop, and streaming progress.
- An MCP server exposing safe tools/resources, routed through a secure gateway.
- Evaluation and observability systems that make quality measurable.
- A capstone AI engineering agent platform with architecture documentation and demo.

2. How to use this plan daily

The plan is built for 2 to 4 hours per day. The minimum path is enough if you are consistent and actually build. The 4-hour path adds deeper reading, refactoring, tests, and deployment polish. Do not compensate for missed days by binge-watching videos. Resume with code.

Daily time	Use this split
2 hours minimum	25 min reading, 85 min building, 10 min notes.
3 hours standard	35 min reading, 115 min building, 30 min tests/docs/notes.
4 hours deep	45 min reading, 150 min building, 45 min refactor/evals/docs.

Daily execution rules

- Every day ends with a Git commit, even if the commit is notes, tests, or a failed experiment summary.
- Every week ends with a demo artifact. If there is no runnable artifact, the week is incomplete.
- Write failure notes. AI engineering skill comes from debugging hallucinations, bad retrieval, looping agents, broken tool calls, cost spikes, and unsafe behavior.
- Prefer official docs and source code over influencer tutorials.
- Do not learn frameworks before understanding the primitives they wrap.
- Never ship an agent feature without tool limits, logging, approval rules, and evals.

Weekly rhythm

Day pattern	Purpose
Day 1	Concept and architecture. Understand the mental model before coding.
Day 2-3	Implement the core primitive. Keep it small and testable.
Day 4-5	Integrate into your app stack and add reliability.
Day 6	Harden, test, document, and add evals/traces where relevant.
Day 7	Review, demo, compress notes, and identify gaps.

3. Required setup and stack

You should not abandon MERN. Keep your product-engineering stack, but add Python and AI infrastructure tools. Use TypeScript where product/UI/backend speed matters. Use Python where AI ecosystem depth matters.

Area	Recommended choices	Why
Frontend	React / Next.js	You already know this. Use it for chat UI, traces, dashboards, approval screens.
Backend	Node/TypeScript plus FastAPI Python service	Node keeps you productive; Python gives access to AI tooling and experiments.
Database	Postgres + pgvector	Relational data plus vector search in one familiar system.
Vector DB	Qdrant as a dedicated option	Useful for hybrid/vector experiments and production patterns.
Cache/queue	Redis + BullMQ or Python equivalent	Needed for jobs, ingestion, long-running agents, rate limits.
Agents	Custom loop first, then LangGraph, then OpenAI Agents SDK experiments	Concept first, framework second.
MCP	Official MCP SDK in TypeScript or Python	Protocol-level tool ecosystem skill.
Observability	OpenTelemetry/OpenInference, Phoenix or LangSmith style tooling	Debugging AI systems needs traces, not logs alone.
Deployment	Docker, managed Postgres, worker process, logs, secrets	Keep deployment simple but real.

Accounts and keys

- OpenAI API account for Responses API, structured outputs, tools, embeddings, and agents experiments.
- Anthropic account if you want Claude and MCP-heavy workflows.
- GitHub account for portfolio repositories and optional GitHub API tools.
- A deployment target such as Render, Fly.io, Railway, VPS, AWS, GCP, Azure, or similar.
- Do not hardcode keys. Use environment variables, secret managers, and .env.example only.

Repo structure

Folder	Purpose
apps/web	Next.js UI for chat, traces, eval dashboard, approvals, capstone demo.
apps/api-node	TypeScript API gateway, auth, routing, tool registry, UI-facing endpoints.
apps/ai-python	FastAPI service, ingestion, evals, LangGraph experiments, Python tooling.
packages/shared	Shared schemas, types, config, validation.
packages/tool-gateway	Policy, tool execution, audit, approval, MCP adapters.
datasets/evals	Golden datasets, RAG eval cases, red-team prompts.
docs	Architecture notes, diagrams, threat models, weekly demos.

4. Source material strategy

Use docs first, short courses second, books/deep resources third. Courses are useful only when they accelerate a project you are actively building. The main curriculum below tells you what to build each day; the resources here tell you where to read when you hit that topic.

Primary sources by topic

Topic	Primary resources
LLM APIs and structured outputs	OpenAI Platform Docs [R1], Responses API [R2], Structured Outputs [R3].
Agents and orchestration	OpenAI Agents SDK [R4], LangGraph docs [R8], LangGraph repository [R9].
MCP	Anthropic MCP announcement [R5], MCP official docs [R6], MCP security docs [R7].
TypeScript AI apps	Vercel AI SDK docs [R10]. Use it after you understand raw SDK calls.
RAG and data frameworks	LlamaIndex RAG docs [R11], Qdrant hybrid search docs [R12].
Evaluation and observability	Ragas docs [R13], OpenInference/Phoenix docs [R14].
Security	OWASP LLM Top 10 [R15], OWASP Agentic Top 10 [R16], MCP security docs [R7].
Production AI engineering	Full Stack Deep Learning [R17].
Short courses	DeepLearning.AI LangGraph [R18], DeepLearning.AI Agentic RAG [R19], Hugging Face Agents Course [R20].

How to avoid tutorial hell

- For every hour of video/course, do at least two hours of implementation.
- Never copy a tutorial project unchanged. Rebuild it inside your own monorepo and connect it to your own architecture.
- Keep a failure log. The failure log is more valuable than polished notes.
- Every concept must become a runnable feature, test, eval, or architecture diagram.
- If a framework hides what is happening, rebuild the primitive manually once.

5. Six-month phase map

Phase	Weeks	Main outcome	Must-have artifact
Phase 1 - LLM foundations	1-4	Direct API usage, structured outputs, tool calling, streaming, reliability.	AI API skeleton and ticket triage assistant.
Phase 2 - RAG systems	5-9	Embeddings, ingestion, retrieval, hybrid search, evals, repo/document assistant.	Repo-aware RAG assistant with evaluation report.
Phase 3 - Agents	10-14	Custom agent loop, LangGraph, research agent, coding agent, multi-agent patterns.	Agent runtime and AI engineering assistant.
Phase 4 - MCP and security	15-18	MCP server, tool suite, secure execution, policy gateway, audit trails.	Secure MCP server routed through tool gateway.
Phase 5 - Production AI infra	19-23	Queues, tracing, evals, model routing, deployment, SRE basics.	Production-shaped AI backend with observability and evals.
Phase 6 - Capstone and Level 3 foundations	24-26	Agent platform architecture and capstone.	AI engineering agent platform case study and demo.

Month 3 checkpoint

By the end of Month 3, you should be able to build a RAG-backed tool-using agent from scratch, explain why it works, show traces, evaluate failures, and rebuild it in LangGraph. This is the foundation checkpoint. If you cannot do this without following a tutorial, repeat Weeks 8 to 14 before moving heavily into MCP and infra.

Month 6 checkpoint

By the end of Month 6, you should have a serious capstone showing Level 2 strength and Level 3 direction. You should be able to discuss platform architecture, tool gateway design, policies, evals, traces, cost/latency, deployment, and security tradeoffs with senior engineers.

6. Day-by-day curriculum

The daily plan below covers 182 days. Treat Day 7 of each week as active review, not a passive rest day. If life interrupts, do not restart the whole plan. Resume from the missed day and protect consistency.

Week 1: AI engineering mental model, environment, and baseline app

Month 1 - LLM foundations and AI engineering setup

Goal	Move from chatbot thinking to software-system thinking: request, context, inference, tools, output, eval, deployment.
Outcome	A clean monorepo with a TypeScript LLM backend, a Python AI service, basic UI, logging, and a daily notes repo.
Core topics	AI-native architecture; LLM request lifecycle; OpenAI/Anthropic SDK basics; Python for an experienced JS developer; Repo discipline and daily commits

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 1	Orientation and target architecture	Read the curriculum overview, define your 6-month goal, create a GitHub monorepo, and write a one-page baseline explaining what you currently know about LLM apps, RAG, agents, and MCP.	Deliver a repo with README, learning log, and a diagram of UI -> API -> LLM -> tools -> data. Commit the baseline honestly.	Convert the diagram into a Mermaid or Excalidraw system map.
Day 2	Environment setup	Install Node LTS, Python 3.12+, uv or poetry, Docker, Postgres, Redis, and a vector DB option. Configure API keys through environment variables only.	Run a Node health endpoint and a Python health endpoint locally. Add reproducible setup commands to README.	Add Docker Compose for Postgres, Redis, and Qdrant.
Day 3	First LLM request	Use the OpenAI or Anthropic SDK directly. Send a simple request, capture request/response metadata, and print token/cost/latency fields where available.	Build a /chat endpoint that accepts user input and returns the model answer with latency and trace ID.	Add streaming response support from backend to frontend.
Day 4	Python bridge for MERN developers	Learn Python syntax only where it matters for AI engineering: virtual environments, async, typing, pydantic, FastAPI, file IO, JSON, and HTTP clients.	Build a FastAPI service with /health, /extract, and /summarize mock endpoints. Call it from Node.	Add typed request/response models and input validation.
Day 5	Minimal chat UI	Build a simple React/Next.js UI for chat, response streaming, model choice, and debug metadata display. Keep it ugly but functional.	The UI can call your Node endpoint, stream output, and show trace ID, latency, and model used.	Persist messages in Postgres with a conversation table.
Day 6	Logging and repo hygiene	Add structured logging, error handling, .env.example, npm scripts, Python scripts, linting, and a simple integration test.	One command should start the stack. One command should run tests. Document both.	Add GitHub Actions for lint and test.
Day 7	Review and compression	Rebuild the week from memory. Write a short architecture note: what is the LLM doing, what is normal software doing, and where failures can happen.	Submit Week 1 demo: chat works, logs work, README is usable, and notes identify 5 gaps to fix next week.	Record a 5-minute screen walkthrough for yourself.

End-of-week evidence

- Runnable artifact for Week 1 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 2: Prompting as runtime design and structured outputs

Month 1 - LLM foundations and AI engineering setup

Goal	Stop treating prompts as magic text. Treat prompts as part of a deterministic runtime contract.
Outcome	A structured extraction and classification service that validates model output against schemas.
Core topics	System and developer instructions; Context boundaries; JSON schema; Structured outputs; Failure cases and retries

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 8	Prompt anatomy	Study instruction hierarchy, task framing, context placement, output constraints, examples, and refusal/fallback handling. Compare a weak prompt and a production prompt.	Create a prompt test file with 10 examples: 5 easy, 3 ambiguous, 2 adversarial.	Add prompt versioning to your repo.
Day 9	Structured extraction	Use schema-based structured output to extract entities from invoices, job posts, or support tickets. Validate output before saving.	Build /extract-structured that returns validated JSON and rejects invalid outputs.	Add retry with a repair prompt only after schema validation fails.
Day 10	Classification and routing	Build a classifier for ticket priority, topic, owner, and required action. Keep classes explicit and measurable.	Create a dataset of 50 synthetic tickets and run batch classification with an accuracy report.	Add confidence estimates and manual review threshold.
Day 11	Streaming UX and cancellation	Implement streaming with cancellation, timeout, and user interruption. Understand how streaming changes frontend state and backend cleanup.	User can stop generation without corrupting conversation state.	Add abort propagation from browser to Node to provider call.
Day 12	Prompt injection basics	Create examples where user text tries to override instructions. Learn why instruction separation and data labeling matter.	Add tests proving external data is treated as untrusted content, not instructions.	Write a short threat model for your chat endpoint.
Day 13	Evaluation by examples	Build a tiny eval runner that sends inputs, compares outputs to expected shape/labels, and records pass/fail.	Run 50 examples and produce a markdown report with failures grouped by category.	Add a regression gate in CI for schema and classifier tests.
Day 14	Weekly consolidation	Refactor prompts, schemas, tests, and docs. Write down prompt patterns that worked and failed.	Week 2 demo: structured extraction, classifier, streaming, eval report, and prompt notes.	Make a dashboard page that lists eval results.

End-of-week evidence

- Runnable artifact for Week 2 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 3: Tool calling and function execution

Month 1 - LLM foundations and AI engineering setup

Goal	Understand the core primitive behind agents: the model decides to call a tool, your code executes it safely.
Outcome	A tool-calling backend with typed tool registry, validation, execution logs, and user-visible results.
Core topics	Function calling; Tool schemas; Tool registry; Execution boundaries; Tool result formatting

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 15	Tool calling concept	Study how tool definitions expose capabilities to a model. Build a mental model: model proposes tool call, app validates, app executes, app returns tool result.	Write 5 tool specs: get_weather_mock, search_docs_mock, calculate, create_ticket_mock, get_user_profile_mock.	Represent tool specs as typed TypeScript objects.
Day 16	Typed tool registry	Build a registry where tools have name, description, input schema, executor, timeout, and permission label.	The LLM can call calculator and search mock tools through your registry.	Add zod validation for every tool input.
Day 17	Real external API tool	Connect one safe real API or internal mock API. Handle rate limits, 4xx/5xx errors, and malformed input.	Create a tool that fetches data and returns a compact result suitable for the model.	Add caching for repeated tool calls.
Day 18	Tool result design	Learn how tool output affects model reasoning. Compare raw JSON, summarized JSON, and pre-filtered results.	Run 20 test prompts and document which tool-result format creates better answers.	Add a max-output-size guard to avoid context bloat.
Day 19	Approval flow	Add human approval before any tool that writes, deletes, spends money, sends email, or executes code.	Create a pending_tool_calls table and approval UI.	Add replay protection so an approved tool call cannot be executed twice.
Day 20	Tool observability	Log tool call arguments, duration, status, output size, and user/session IDs. Do not log secrets.	Produce a tool-call timeline for one conversation.	Add a basic trace viewer in the UI.
Day 21	Weekly consolidation	Refactor the tool system and write a guide: how to add a new tool, test it, and secure it.	Week 3 demo: model calls at least 3 tools, approval works, logs are visible, and tests pass.	Build the same tool registry in Python to compare ergonomics.

End-of-week evidence

- Runnable artifact for Week 3 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 4: Reliability, guardrails, cost, latency, and mini project

Month 1 - LLM foundations and AI engineering setup

Goal	Make LLM-backed features behave like software systems that can be tested, observed, and operated.
Outcome	A production-shaped AI API skeleton with retries, fallbacks, rate limiting, cost tracking, and a demo app.
Core topics	Timeouts; Retries; Fallbacks; Rate limits; Cost estimation; Guardrails; Production API shape

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 22	Failure taxonomy	List failures: provider timeout, bad JSON, unsafe request, hallucinated tool use, high latency, excessive cost, rate limit, prompt injection, empty output.	Create an error taxonomy and map each error to retry, fail, fallback, or human review.	Add error codes to API responses.
Day 23	Retries and fallbacks	Implement retry only for safe failure modes. Add model fallback for transient provider failures.	Run tests proving schema failures, rate limits, and timeouts are handled differently.	Add circuit breaker behavior after repeated provider failures.
Day 24	Cost and latency budget	Track tokens, model, duration, and estimated cost per request. Define limits per endpoint.	Create a cost report for 100 sample calls.	Add budget-based routing: cheap model for simple tasks, stronger model for hard tasks.
Day 25	Guardrails and output validation	Add input validation, schema validation, refusal handling, output size limits, and unsafe tool protection.	Every endpoint has a validation layer before model call and before response persistence.	Add a simple policy engine for tool permissions.
Day 26	Mini project build	Build an AI ticket triage assistant: classify ticket, extract fields, propose next action, and optionally create a mock issue after approval.	End-to-end flow works from UI through API to database and tool registry.	Add admin page for failed requests and manual review.
Day 27	Mini project hardening	Add tests, seed data, screenshots, README, architecture diagram, and known limitations.	The project can be run by another developer using README only.	Deploy to a low-cost environment.
Day 28	Month 1 review	Review the whole month. Rebuild key pieces from memory: structured output, tool call, streaming, eval runner, approval.	Month 1 checkpoint: foundation correct if you can explain and demo all primitives without a tutorial.	Write a blog-style internal note: LLM features are distributed systems with probabilistic components.

End-of-week evidence

- Runnable artifact for Week 4 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 5: Embeddings, vector search, and retrieval basics

Month 2 - RAG and knowledge systems

Goal	Understand how private data enters an LLM system without training a model.
Outcome	A working document search service using embeddings, metadata, and vector search.
Core topics	Embeddings; Vector similarity; Chunking; pgvector; Qdrant; Metadata filtering

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 29	RAG mental model	Study the RAG pipeline: load, parse, chunk, embed, index, retrieve, rerank, synthesize, cite, evaluate.	Draw the full RAG architecture and annotate where quality can fail.	Compare RAG with fine-tuning in a one-page note.
Day 30	Embeddings playground	Generate embeddings for short texts and inspect nearest neighbors. Learn cosine similarity and why semantic similarity is not truth.	Create a script that embeds 100 snippets and queries similar snippets.	Visualize embeddings in 2D using a simple projection.
Day 31	Vector DB setup	Use pgvector or Qdrant. Create collections/tables with text, embedding, source, metadata, and timestamps.	Store and query chunks from 10 markdown files.	Add metadata filters by source, author, and date.
Day 32	Chunking experiments	Compare fixed-size chunks, heading-aware chunks, and overlap strategies. Measure retrieval quality manually.	Create a chunking report with examples of good and bad chunks.	Implement a pluggable chunker interface.
Day 33	Basic RAG answerer	Retrieve top chunks, build a context block, ask the model to answer only from context, and return citations.	Build /ask-docs endpoint with answer, cited chunks, and no-answer behavior.	Add streaming answer with cited sources shown before final answer.
Day 34	Retrieval tests	Create 30 question-answer pairs from your documents. Track whether retrieval found the correct chunk.	Produce retrieval@k measurements manually or with a simple script.	Add failed-query analysis to learning notes.
Day 35	Weekly consolidation	Refactor ingestion and retrieval. Document chunking decisions and known retrieval failure modes.	Week 5 demo: upload docs, index docs, ask questions, receive cited answers.	Swap vector DB and compare implementation effort.

End-of-week evidence

- Runnable artifact for Week 5 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 6: Data ingestion pipelines for docs, PDFs, web pages, and code

Month 2 - RAG and knowledge systems

Goal	Build ingestion that is reliable enough for messy real-world knowledge.
Outcome	A reusable ingestion pipeline with parsers, chunkers, dedupe, metadata, and reindexing.
Core topics	Document loaders; Parsing; Code-aware chunking; Deduplication; Incremental indexing; Source metadata

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 36	Ingestion architecture	Design loader -> parser -> cleaner -> chunker -> embedder -> indexer -> verifier. Keep each stage testable.	Create pipeline interfaces and a CLI command to ingest a folder.	Add a job table for ingestion status.
Day 37	Markdown and text ingestion	Implement stable ingestion for markdown, txt, and JSON. Preserve headings and source paths.	Index 50 files and confirm metadata is searchable.	Add content hash dedupe.
Day 38	PDF ingestion	Parse PDFs carefully and capture page numbers. Identify where extraction fails and where OCR would be needed.	Index 3 PDFs with page-level citations.	Add a parser quality score per document.
Day 39	Web page ingestion	Fetch public pages, clean boilerplate, respect source URL, and chunk by headings.	Index 10 web pages and answer questions with URL citations.	Add robots/respectful-fetch notes to README.
Day 40	Codebase ingestion	Chunk code by file, function/class boundaries where possible, and keep path, language, exports, imports, and repo metadata.	Index one MERN repo and ask codebase questions.	Add separate retrieval modes for docs and code.
Day 41	Incremental reindexing	Detect changed files, deleted files, and unchanged files. Avoid re-embedding everything.	Run ingestion twice and prove unchanged files are skipped.	Add reindex queue with retry.
Day 42	Weekly consolidation	Write a pipeline design doc and run ingestion against a realistic mixed corpus.	Week 6 demo: folder ingestion, PDF ingestion, code ingestion, incremental reindexing.	Add admin UI for ingestion jobs.

End-of-week evidence

- Runnable artifact for Week 6 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 7: Retrieval quality, hybrid search, reranking, and query transformation

Month 2 - RAG and knowledge systems

Goal	Move beyond naive top-k vector search. Most production RAG depends on retrieval quality.
Outcome	A RAG system with hybrid retrieval, filters, reranking, query rewriting, and measurable retrieval quality.
Core topics	BM25; Hybrid search; Reranking; Query expansion; Multi-query retrieval; Metadata-aware retrieval

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 43	Why naive RAG fails	Study failure modes: wrong chunk, missing chunk, too many chunks, ambiguous query, stale source, permission mismatch, and poor synthesis.	Write 20 failure examples against your existing system.	Categorize failures as ingestion, retrieval, ranking, prompt, or data issue.
Day 44	Keyword + vector retrieval	Add keyword/BM25 or database full-text search alongside vector search.	Compare vector-only vs keyword-only vs hybrid on 30 questions.	Implement reciprocal-rank-fusion style merging.
Day 45	Reranking	Add a rerank step using a reranker model or provider capability. Keep input size controlled.	Measure whether reranking improves top-3 relevance.	Add rerank cost and latency tracking.
Day 46	Query transformation	Implement query rewrite, multi-query generation, and acronym expansion. Do not blindly trust generated queries.	Run query transformation on ambiguous codebase questions.	Add a toggle to compare raw vs rewritten query.
Day 47	Metadata filters and permissions	Add filters by tenant, repo, file type, date, owner, and permission group.	Prove a user cannot retrieve chunks outside their permission boundary.	Add test fixtures for cross-tenant leakage.
Day 48	Answer synthesis	Improve prompt design: answer from context, cite each claim, say insufficient evidence when needed, avoid over-answering.	Run an answer quality eval over 30 questions.	Add citation coverage score.
Day 49	Weekly consolidation	Create a retrieval quality dashboard with failure categories, top-k hit rate, latency, and cost.	Week 7 demo: hybrid retrieval, reranking, query rewriting, metadata filters.	Write a deep note: retrieval is the real product, the model is the final formatter.

End-of-week evidence

- Runnable artifact for Week 7 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 8: RAG evaluation, hallucination control, and repo-aware assistant

Month 2 - RAG and knowledge systems

Goal	Build confidence that your RAG system answers correctly and refuses when context is insufficient.
Outcome	A repo-aware assistant with citations, evaluation dataset, failure reports, and regression tests.
Core topics	Golden datasets; Retrieval evals; Answer evals; Citation checks; Repo-aware QA; Hallucination control

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 50	Golden dataset design	Create a small but serious evaluation set: exact-answer questions, summary questions, comparison questions, and unanswerable questions.	Build a dataset with 80 questions and expected evidence IDs.	Add a JSONL format for eval cases.
Day 51	Retrieval evaluation	Measure if correct evidence appears in top-k. Separate retrieval failure from generation failure.	Produce retrieval hit rate at k=3, k=5, and k=10.	Add per-source and per-file-type breakdown.
Day 52	Answer evaluation	Evaluate groundedness, correctness, completeness, and citation coverage. Use human review first, optional LLM judge later.	Review 30 answers and record failure reasons.	Add an LLM-as-judge experiment with human spot check.
Day 53	Repo-aware assistant	Index a real MERN repo. Build features: explain file, trace API flow, find auth logic, answer where a bug should be fixed.	Demo 10 codebase questions with cited files and line/page-like references where possible.	Add code search mode separated from docs search mode.
Day 54	Hallucination controls	Add refusal behavior, answer confidence based on retrieval quality, source requirement, and no-answer path.	System refuses 10 intentionally unanswerable questions.	Add source-aware response templates for explain, compare, and summarize.
Day 55	Regression suite	Run the full eval suite before and after prompt/retrieval changes. Track regressions.	Create a report comparing two versions of your RAG pipeline.	Add CI job for a small smoke eval.
Day 56	Weekly consolidation	Package the repo assistant as a portfolio project with architecture diagram, eval results, and failure analysis.	Week 8 demo: repo assistant with eval report and grounded answers.	Record a demo where you use it to onboard into an unfamiliar repo.

End-of-week evidence

- Runnable artifact for Week 8 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 9: Production RAG: multi-tenancy, caching, freshness, and summarization

Month 2 - RAG and knowledge systems

Goal	Turn RAG from a demo into a production-shaped service.
Outcome	A multi-tenant RAG backend with permissions, update flows, caching, summarization, and monitoring.
Core topics	Multi-tenancy; Access control; Freshness; Caching; Summarization; Data lifecycle

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 57	Multi-tenant design	Design tenant, workspace, user, source, document, chunk, embedding, job, and query tables. Identify permission checks.	Write schema migrations or ERD for multi-tenant RAG.	Add row-level security or app-level permission tests.
Day 58	Freshness and reindexing	Track document modified time, hash, index version, embedding model version, and stale chunks.	Implement reindex for changed sources and cleanup for deleted documents.	Add index versioning to allow rollback.
Day 59	Caching	Cache embeddings, retrieval results, model responses where safe, and rerank outputs. Know what must not be cached.	Add cache hit/miss metrics and invalidation rules.	Add semantic cache experiment for repeated questions.
Day 60	Summarization layer	Build document summaries, folder/source summaries, and conversation summaries. Understand summary drift.	Create summaries with source links and timestamps.	Add a summary refresh job when underlying docs change.
Day 61	Observability for RAG	Log query, rewritten query, retrieved chunk IDs, scores, rerank scores, answer, citations, cost, latency, and user feedback.	Create a query trace view for one answer.	Add thumbs-up/down feedback tied to trace ID.
Day 62	Month 2 hardening	Run security checks for permissions, prompt injection through retrieved docs, oversized chunks, and stale sources.	Show tests for cross-tenant leakage and injected document text.	Add a red-team dataset of malicious documents.
Day 63	Month 2 review	Consolidate all RAG learnings into one architecture document. Compare your system against naive RAG.	Month 2 checkpoint: production-shaped repo/document assistant with evals and security tests.	Write a case study: RAG quality comes from data and retrieval engineering.

End-of-week evidence

- Runnable artifact for Week 9 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 10: Agents from scratch: loops, state, tools, and memory

Month 3 - Agents and orchestration

Goal	Understand agents without hiding behind frameworks.
Outcome	A custom single-agent runtime with tool loop, state, memory, max steps, and stop conditions.
Core topics	Agent loop; Planning vs acting; State machine; Memory; Stop conditions; Tool registry reuse

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 64	Agent definition	Define agent precisely: model plus instructions plus state plus tools plus loop plus stopping rules plus memory plus policy.	Write an agent runtime design doc with sequence diagram.	Compare chatbot, workflow, and agent in a table.
Day 65	Build the loop	Implement a loop: send state to model, receive message or tool call, validate tool call, execute tool, append result, repeat until final or max steps.	Agent can use calculator and search mock tools over multiple steps.	Add step-by-step trace display.
Day 66	State and memory	Separate short-term state, conversation memory, long-term memory, and external knowledge. Do not dump everything into context.	Add conversation summary memory and user preference memory.	Add memory write approval to prevent accidental persistence.
Day 67	Planning strategies	Compare no-plan, plan-first, plan-and-execute, and ReAct-style loops. Keep plans inspectable.	Run the same 10 tasks with two planning modes and compare failures.	Add plan revision after tool failure.
Day 68	Safety limits	Add max steps, max tool calls, max cost, max execution time, tool allowlist, and dangerous action approval.	Demonstrate a task that stops safely instead of looping forever.	Add loop-detection based on repeated tool calls.
Day 69	Agent task: research assistant	Build a simple research assistant that searches docs/web mock, extracts evidence, drafts answer, and cites sources.	Agent completes 5 research tasks with trace and citations.	Add critique pass before final answer.
Day 70	Weekly consolidation	Write down every failure mode observed: bad plan, bad tool input, wrong memory, looping, weak evidence, overconfident answer.	Week 10 demo: custom agent runtime with trace, tools, memory, limits.	Refactor runtime into reusable package.

End-of-week evidence

- Runnable artifact for Week 10 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 11: LangGraph and stateful workflows

Month 3 - Agents and orchestration

Goal	Learn controllable agent workflows: graphs, state, checkpoints, branching, and human-in-the-loop.
Outcome	A LangGraph workflow that persists state, supports retries, streams progress, and allows approval.
Core topics	StateGraph; Nodes and edges; Conditional routing; Checkpoints; Human-in-loop; Durable execution

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 71	LangGraph basics	Study graph concepts: node, edge, state, reducer, conditional route, checkpoint. Map your custom loop into graph terms.	Build a two-node graph: classify request -> answer.	Add graph visualization if possible.
Day 72	Tool-using graph	Create nodes for plan, retrieve, tool_execute, synthesize, and final. Keep state explicit.	Run a RAG agent workflow through graph nodes.	Add node-level timing and errors.
Day 73	Conditional routes	Route based on confidence, tool need, retrieval score, and approval requirement.	Low-confidence queries go to clarification or refusal instead of answer.	Add branch coverage tests.
Day 74	Persistence and resume	Use checkpoints or your own persistence to resume an interrupted workflow.	Start a workflow, interrupt it, resume from saved state.	Add resumable state to the UI.
Day 75	Human-in-the-loop	Pause before risky tool execution or final external action. Allow approve, reject, or edit.	Workflow pauses for approval and resumes after decision.	Add audit records for decisions.
Day 76	Streaming progress	Stream graph progress to UI: current node, events, tool calls, final answer.	User sees live agent progress without exposing secrets.	Add collapsible trace details.
Day 77	Weekly consolidation	Build a durable ticket-resolution agent using LangGraph: classify, retrieve policy, draft answer, ask approval, create ticket.	Week 11 demo: stateful workflow with persistence and approval.	Write a note comparing custom loop vs LangGraph.

End-of-week evidence

- Runnable artifact for Week 11 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 12: Research agents, browser/search tools, and verification

Month 3 - Agents and orchestration

Goal	Build agents that gather evidence instead of making unsupported claims.
Outcome	A research agent that plans, searches, extracts evidence, cross-checks, cites, and refuses weak claims.
Core topics	Research workflow; Search tools; Evidence extraction; Cross-checking; Citation discipline; Verification loops

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 78	Research workflow design	Design research as stages: understand question, generate search plan, gather sources, extract claims, verify, synthesize, cite.	Write a research-agent system design with failure modes.	Add source quality criteria to the design.
Day 79	Search and fetch tools	Build tools for search/fetch or use mock sources if needed. Clean pages and preserve source metadata.	Agent can collect source snippets and store them as evidence objects.	Add source deduplication.
Day 80	Evidence extraction	Extract atomic claims from sources with quote/source/url/date fields. Avoid long copyrighted quotes in outputs.	Build an evidence table for 5 research tasks.	Add claim type: fact, opinion, instruction, code, date-sensitive.
Day 81	Cross-checking	Require important claims to be supported by multiple sources when possible. Mark conflicts clearly.	Agent flags conflicting evidence instead of choosing silently.	Add a contradiction detector prompt or rule.
Day 82	Synthesis and citations	Generate answers from evidence objects, not raw pages. Each major claim should map to source IDs.	Final answer includes citations and uncertainty where evidence is weak.	Add answer sections: findings, evidence, unknowns.
Day 83	Evaluation	Create 20 research tasks and grade evidence quality, answer correctness, citation coverage, and overclaiming.	Produce a research-agent eval report.	Add a verification pass that critiques final answer before user sees it.
Day 84	Weekly consolidation	Package the research agent with trace UI and evidence table.	Week 12 demo: research agent with sources, verification, and clear unknowns.	Use it to research one AI engineering topic you will study next.

End-of-week evidence

- Runnable artifact for Week 12 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 13: Coding agent foundations: files, patches, tests, and sandbox execution

Month 3 - Agents and orchestration

Goal	Understand how coding agents work under the hood and where they are dangerous.
Outcome	A local coding assistant that reads files, proposes patches, runs tests in a sandbox, and reports results.
Core topics	Filesystem tools; Patch generation; Command execution; Sandboxing; Test loops; Repo context

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 85	Coding-agent architecture	Study the loop: inspect repo, choose files, propose changes, apply patch, run tests, inspect failure, iterate.	Write a coding-agent threat model before coding anything.	List tools by risk level: read, write, execute, network.
Day 86	Read-only repo assistant	Build tools: list_files, read_file, search_files, grep, get_package_scripts. Keep path traversal blocked.	Agent answers repo questions using read-only tools.	Add max file size and allowed directory restrictions.
Day 87	Patch proposal	Ask the model to produce a patch plan and unified diff. Do not auto-apply yet.	Agent proposes patches for 3 simple bugs and explains expected tests.	Add diff validation.
Day 88	Apply patch with approval	Implement apply_patch only after approval. Store before/after diff and allow rollback.	Patch can be approved, applied, and reverted.	Add a UI diff viewer.
Day 89	Command execution sandbox	Run safe test commands in a constrained environment. Block arbitrary shell by default.	Agent can run npm test or a whitelisted command and read output.	Add timeouts and output truncation.
Day 90	Debug loop	Let the agent inspect test failure and propose one fix iteration. Keep max iterations low.	Agent fixes a deliberately broken small project.	Add a benchmark of 10 small bugs.
Day 91	Weekly consolidation	Document limitations: why coding agents need sandboxing, approvals, and test grounding.	Week 13 demo: read-only analysis, approved patch, test run, rollback.	Compare your flow with Cursor/Claude Code at a conceptual level.

End-of-week evidence

- Runnable artifact for Week 13 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 14: Multi-agent patterns, handoffs, supervisor architectures, and Month 3 checkpoint

Month 3 - Agents and orchestration

Goal	Understand when multiple agents help and when they create complexity theater.
Outcome	A controlled multi-agent workflow with explicit handoffs, specialist roles, evals, and a Month 3 foundation demo.
Core topics	Supervisor pattern; Agents-as-tools; Handoffs; Specialists; Critic/reviewer; Multi-agent evals

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 92	When multi-agent is useful	Study patterns: supervisor, router, specialist, critic, planner/executor, agents-as-tools. Also study when a normal workflow is better.	Write decision rules for single workflow vs single agent vs multi-agent.	Draw architecture for 3 agent patterns.
Day 93	Supervisor + specialist	Build a supervisor that routes to RAG specialist, coding specialist, or research specialist. Keep tool permissions separate.	Supervisor routes 15 tasks correctly.	Add fallback when specialist confidence is low.
Day 94	Critic/reviewer agent	Add a reviewer that checks grounding, security, completeness, and format before final response.	Reviewer catches at least 5 seeded errors.	Compare self-critique vs separate reviewer.
Day 95	Handoffs and state	Implement handoff with state summary, task goal, constraints, and expected output. Avoid dumping full conversation blindly.	Handoff logs are inspectable and small.	Add handoff schema validation.
Day 96	Multi-agent evals	Create an eval set for routing correctness, final answer quality, cost, and latency.	Measure if multi-agent improves quality enough to justify cost.	Add a no-agent baseline comparison.
Day 97	Month 3 integrated project	Build an AI engineering assistant: can answer repo questions, research docs, propose patch plan, and ask approval for risky actions.	Integrated demo uses RAG, tools, agent loop, graph workflow, and approval.	Deploy a private demo environment.
Day 98	Month 3 review	Review foundations. You should now understand LLM primitives, RAG, agents, workflows, tools, state, memory, and evaluation.	Month 3 checkpoint: you can build AI agents from scratch and with LangGraph, not just use tools.	Write your Level 2 readiness assessment and gaps.

End-of-week evidence

- Runnable artifact for Week 14 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 15: MCP concepts and first server

Month 4 - MCP, tool ecosystems, and security

Goal	Understand MCP as a standard way for AI clients to discover and call external capabilities.
Outcome	A working MCP server exposing safe tools and resources to an MCP client.
Core topics	MCP client/server; Tools; Resources; Prompts; Transports; Capability discovery

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 99	MCP mental model	Study MCP client, server, tools, resources, prompts, transport, and capability discovery. Compare MCP with REST and plugin systems.	Write a one-page explanation of MCP for a MERN developer.	Map your Week 3 tool registry to MCP terms.
Day 100	First MCP server	Build a minimal MCP server in TypeScript or Python exposing calculator and echo-safe tools.	Connect it to a compatible MCP client and call the tools.	Add structured input schemas for all tools.
Day 101	Resources and prompts	Expose read-only resources such as project README, package metadata, or docs snippets. Add prompt templates if supported.	Client can list resources and use them as context.	Add resource pagination or size limits.
Day 102	Transport and lifecycle	Understand STDIO vs HTTP/SSE or streamable HTTP options as applicable. Learn startup, shutdown, logging, and error behavior.	Document how your server starts and how failures appear in the client.	Add health check and version endpoint where relevant.
Day 103	MCP server from existing tools	Wrap your existing tool registry into MCP tools. Keep permission labels and validation.	At least 3 existing tools are available through MCP.	Create a small adapter layer between internal tools and MCP schema.
Day 104	Testing MCP tools	Write tests for tool schemas, invalid inputs, permission failures, and tool output size.	MCP server test suite passes locally.	Add mock client tests.
Day 105	Weekly consolidation	Write setup instructions and a diagram: AI client -> MCP server -> tool gateway -> data/API.	Week 15 demo: functional MCP server with tools/resources and tests.	Connect the server to both a coding client and desktop client if available.

End-of-week evidence

- Runnable artifact for Week 15 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 16: MCP tools for database, GitHub, filesystem, and internal APIs

Month 4 - MCP, tool ecosystems, and security

Goal	Build useful MCP capabilities while keeping reads, writes, and dangerous actions separated.
Outcome	An MCP tool suite that can query safe data, inspect repos, and create approved actions.
Core topics	Database tools; GitHub tools; Filesystem tools; API tools; Permission scopes; Read/write separation

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 106	Tool boundary design	Classify tools: read-only, write with approval, destructive, external side effect, code execution. Design separate namespaces.	Create a tool risk matrix for your MCP server.	Add risk labels to tool descriptions.
Day 107	Database read tools	Expose safe DB read tools with parameterized queries only. Never allow raw SQL from model/user.	Tool can fetch ticket by ID and search tickets by filters.	Add tenant filter enforcement.
Day 108	GitHub/repo tools	Expose read-only repo tools: list issues mock/live, read file, search code, inspect PR metadata.	Client can ask repo questions through MCP tools.	Add rate-limit handling.
Day 109	Filesystem safety	Implement safe filesystem access with root jail, path normalization, file size limits, and read-only default.	Tests prove path traversal does not work.	Add allowlist by file extension.
Day 110	Approved write action	Create one write action such as create_mock_issue or draft_pr_description. Require approval outside the model.	Write action cannot execute without explicit approval token.	Add audit log entry with actor, args, and result.
Day 111	Tool descriptions and UX	Improve tool names/descriptions so models choose correctly. Remove vague tools.	Run 30 prompts and measure correct tool selection.	Add a bad-tool-selection test set.
Day 112	Weekly consolidation	Package tool suite with docs and example prompts. Review all tools for least privilege.	Week 16 demo: DB, repo, filesystem, and approved write tools exposed via MCP.	Create a public-safe demo variant with mock data.

End-of-week evidence

- Runnable artifact for Week 16 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 17: Secure tool execution, prompt injection, sandboxing, and AI security

Month 4 - MCP, tool ecosystems, and security

Goal	Security becomes non-negotiable once agents can act. Learn AI-specific and agent-specific risk.
Outcome	A hardened agent/MCP stack with threat model, policy controls, sandboxing, and red-team tests.
Core topics	OWASP LLM risks; OWASP agentic risks; Prompt injection; Tool poisoning; Sandboxing; Least privilege

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 113	AI security baseline	Study OWASP LLM and Agentic AI risks. Translate each risk into developer controls for your stack.	Create a security checklist for every AI feature you build.	Map controls to code locations in your repo.
Day 114	Prompt injection red-team	Create malicious documents, malicious tool outputs, and malicious user prompts. Test if the agent follows them.	Add red-team tests that your agent must fail safely.	Add instruction/data labeling to all retrieved context.
Day 115	Tool poisoning and rug-pull risks	Understand risks from tool descriptions, changing tool behavior, untrusted MCP servers, and marketplace-style integrations.	Add tool versioning, approval, and pinned trusted tool config.	Hash tool definitions and alert on changes.
Day 116	Sandboxing execution	Isolate any command/code execution. Restrict network, filesystem, environment variables, time, memory, and allowed commands.	Dangerous command requests are blocked and logged.	Run test commands in a disposable container.
Day 117	Secrets and data protection	Prevent model access to secrets. Scrub logs. Avoid sending sensitive data unnecessarily to model providers.	Add secret scanning and log redaction tests.	Add data classification labels to tool outputs.
Day 118	Authorization and approval	Design auth for human users, agents, and tools. Enforce user permissions before tool execution.	Write tests for unauthorized tool calls and cross-tenant access.	Add approval UI with reason and diff preview.
Day 119	Weekly consolidation	Produce a threat model for your MCP + agent platform: assets, actors, trust boundaries, attacks, controls, residual risk.	Week 17 demo: red-team tests, sandbox, approvals, secret redaction, threat model.	Ask another developer to review your threat model.

End-of-week evidence

- Runnable artifact for Week 17 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 18: Agent runtime, tool gateway, policies, and audit trails

Month 4 - MCP, tool ecosystems, and security

Goal	Think like infrastructure: agents should run through a controlled gateway, not direct random tools.
Outcome	A policy-enforced tool gateway for agents and MCP servers with audit logs and approvals.
Core topics	Tool gateway; Policy engine; Audit trail; Execution queue; Approval workflows; Runtime boundaries

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 120	Tool gateway design	Design a gateway that all agent tools go through: validate, authorize, approve, execute, log, return.	Create gateway sequence diagram and API contract.	Add policy decision point and policy enforcement point vocabulary.
Day 121	Policy engine	Implement simple policies: user role, tenant, tool risk, environment, time budget, data classification, approval requirement.	Policy denies risky calls automatically and explains why.	Use a declarative policy config file.
Day 122	Audit logging	Record actor, user, tenant, tool, args hash, approval, result summary, duration, cost, and trace ID.	Audit log can reconstruct an agent action timeline.	Add immutable append-only log table pattern.
Day 123	Queued execution	Move slow/risky tool execution to a job queue. Support pending, running, succeeded, failed, cancelled.	Agent receives job status and final result through polling or event stream.	Add retry policies per tool.
Day 124	Runtime isolation	Separate model orchestration, tool gateway, data services, and UI. Avoid one giant backend.	Refactor services or modules to clear boundaries.	Add service-level README for each boundary.
Day 125	Integrated MCP gateway	Route MCP tool calls through your gateway. Do not let MCP handlers directly execute dangerous actions.	MCP tools now inherit auth, policy, audit, and approval.	Add smoke tests from MCP client through gateway.
Day 126	Month 4 review	Review MCP, security, gateway, and infra concepts. Update your portfolio diagrams.	Month 4 checkpoint: you can build and secure MCP servers, not just connect random ones.	Write a paper-style note: safe agent systems need infrastructure, not vibes.

End-of-week evidence

- Runnable artifact for Week 18 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 19: Async jobs, queues, workers, retries, and long-running agents

Month 5 - Production AI systems and infra

Goal	Production agents are long-running workflows. Learn job orchestration like an infra engineer.
Outcome	A worker-based AI backend supporting long-running tasks, cancellation, retries, idempotency, and progress events.
Core topics	Queues; Workers; Idempotency; Retries; Cancellation; Progress streaming; Long-running workflows

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 127	Async architecture	Design which work stays synchronous and which becomes queued: indexing, evals, long agent tasks, batch extraction, report generation.	Create a queue architecture diagram.	Choose BullMQ, Celery, Temporal, or equivalent for experiments.
Day 128	Basic worker	Implement queue + worker for document ingestion or long extraction. Track job status in DB.	User can submit job and view status.	Add progress events.
Day 129	Retries and idempotency	Add retry rules, idempotency keys, dedupe, and safe resume. Understand why retries can duplicate side effects.	Prove repeated job submission does not duplicate documents/actions.	Add exactly-once-like behavior where feasible.
Day 130	Cancellation and timeouts	Support cancellation for long model calls, tool calls, and workflows. Persist cancelled state.	User can cancel a running job and see partial trace.	Add cleanup logic for temporary files.
Day 131	Long-running agent workflow	Run an agent workflow through the queue. Store each step and allow resume after crash.	Agent survives process restart or can resume from last checkpoint.	Add watchdog for stuck jobs.
Day 132	Backpressure and rate limits	Control concurrency by tenant, tool, model, and queue. Prevent runaway agents.	Load test with many jobs and show queue metrics.	Add per-tenant quotas.
Day 133	Weekly consolidation	Document worker architecture, retries, idempotency, and failure handling.	Week 19 demo: queued agent task with progress, cancellation, retry, and resume.	Compare queue-based orchestration with graph checkpointing.

End-of-week evidence

- Runnable artifact for Week 19 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 20: Observability, tracing, debugging, and production feedback loops

Month 5 - Production AI systems and infra

Goal	You cannot operate AI systems you cannot inspect.
Outcome	A traceable AI system with spans for model calls, retrieval, tools, prompts, outputs, costs, and user feedback.
Core topics	Tracing; OpenTelemetry; OpenInference; Phoenix/LangSmith; Prompt/version tracking; Feedback loops

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 134	Observability model	List what to observe: prompt version, model, input size, output size, cost, latency, retrieval IDs, tool calls, errors, eval scores, feedback.	Create an observability schema for your AI platform.	Add PII redaction rules before traces leave app.
Day 135	Tracing integration	Instrument model calls, retrieval, tool gateway, and agent steps with trace IDs and spans.	One user request produces a full trace tree.	Export traces to an observability backend.
Day 136	Prompt and model versioning	Track prompt templates, model versions, tool versions, and retrieval config per request.	A trace can tell exactly which config produced an answer.	Add config diff view between two runs.
Day 137	Debugging failed runs	Create a workflow for debugging: inspect trace, classify failure, reproduce with same inputs, patch, rerun eval.	Debug 5 real failures from your eval logs.	Add run replay feature for saved traces.
Day 138	User feedback	Capture thumbs, correction text, expected answer, and issue labels. Tie feedback to traces and eval datasets.	User feedback can create a new eval case.	Add feedback triage dashboard.
Day 139	Operational metrics	Create metrics: p50/p95 latency, cost per task, failure rate, refusal rate, tool failure rate, retrieval hit rate, approval rate.	Generate weekly ops report from your local data.	Add alert thresholds for runaway cost or high failure rate.
Day 140	Weekly consolidation	Add a debugging playbook and trace screenshots to your docs.	Week 20 demo: full trace, replay, feedback-to-eval, ops metrics.	Integrate Phoenix, LangSmith, or equivalent as an experiment.

End-of-week evidence

- Runnable artifact for Week 20 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 21: Evaluation systems: golden sets, regression gates, RAG evals, and agent evals

Month 5 - Production AI systems and infra

Goal	Make quality measurable enough to ship changes without guessing.
Outcome	An evaluation harness for prompts, models, retrieval configs, tools, agents, and cost/latency tradeoffs.
Core topics	Eval datasets; Regression tests; LLM-as-judge; Human review; RAG metrics; Agent trajectory evals

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 141	Eval strategy	Define what quality means for each feature: exactness, groundedness, task success, safety, speed, cost, user satisfaction.	Create an eval plan for chat, RAG, and agents.	Classify evals as unit, integration, offline, online.
Day 142	Golden datasets	Build small high-quality datasets instead of giant noisy ones. Include edge cases and unanswerable cases.	Create 150 total eval cases across extraction, RAG, and agent tasks.	Tag cases by difficulty and risk.
Day 143	Automated eval runner	Build a runner that executes cases against a chosen config and saves outputs, traces, scores, cost, and latency.	Run baseline config and save results.	Add comparison report between two configs.
Day 144	LLM-as-judge carefully	Use judge models for rubric-based scoring, but calibrate with human review and examples.	Create rubrics for groundedness, helpfulness, citation quality, and safety.	Measure judge disagreement on 20 cases.
Day 145	Agent trajectory evals	Evaluate not just final answer but steps: correct tool choice, safe args, no unnecessary calls, correct stopping.	Score 30 agent traces.	Add failure labels: planning, tool selection, execution, synthesis, safety.
Day 146	CI regression gate	Add a small smoke eval to CI and a larger nightly/local eval. Define pass thresholds.	A bad prompt change fails the eval gate.	Add cost budget to eval runs.
Day 147	Weekly consolidation	Create an evaluation dashboard or report folder with trend charts.	Week 21 demo: eval harness, regression report, judge rubric, agent trajectory scoring.	Add Ragas or another RAG evaluation tool as a comparative experiment.

End-of-week evidence

- Runnable artifact for Week 21 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 22: Model routing, cost control, latency engineering, and reliability patterns

Month 5 - Production AI systems and infra

Goal	Learn to choose models and runtime paths based on task difficulty, cost, latency, and risk.
Outcome	A router that selects model/workflow/tool path and tracks cost-quality tradeoffs.
Core topics	Model routing; Fallbacks; Caching; Batching; Latency budgets; Cost budgets; Quality tiers

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 148	Routing principles	Define routing by task type, difficulty, data sensitivity, latency need, cost budget, and required reasoning depth.	Create routing matrix for extraction, chat, RAG, agent, and code tasks.	Add a cheap/standard/deep mode flag.
Day 149	Implement model router	Build a router abstraction that supports provider, model, temperature, timeout, max tokens, fallback, and logging.	Endpoints call router instead of provider SDK directly.	Add provider failover simulation.
Day 150	Cost controls	Add per-user and per-tenant daily budgets, max tokens, max steps, and tool-call limits.	Requests beyond budget degrade gracefully or require approval.	Add budget dashboard.
Day 151	Latency controls	Measure p95. Add streaming, parallel retrieval, timeout budgets, early exits, smaller context, and cached summaries.	Reduce latency on one endpoint by at least 30 percent without major quality loss.	Add latency breakdown by component.
Day 152	Caching and batching	Cache safe repeated operations and batch embeddings/evals. Understand invalidation and privacy risks.	Embedding batch job is faster and cheaper than one-by-one calls.	Add cache policy table to docs.
Day 153	Quality-cost experiments	Run evals across model choices and retrieval configs. Choose default based on quality per rupee/dollar and latency.	Produce a model/config comparison report.	Add routing rule based on eval findings.
Day 154	Weekly consolidation	Document model routing architecture and operational tradeoffs.	Week 22 demo: router, budgets, caching, latency improvements, comparison report.	Add LiteLLM or provider abstraction experiment if useful.

End-of-week evidence

- Runnable artifact for Week 22 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 23: Deployment architecture, multi-tenancy, reliability, and SRE basics for AI systems

Month 5 - Production AI systems and infra

Goal	Think like the person responsible for running this in production.
Outcome	A deployable AI backend with separation of services, secure config, monitoring, and rollback plan.
Core topics	Docker; Service boundaries; Secrets; Migrations; Rollbacks; Multi-region basics; SLOs

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 155	Production architecture	Design services: UI, API, AI orchestrator, worker, vector DB, relational DB, cache, observability, tool gateway.	Create final production architecture diagram.	Identify single points of failure.
Day 156	Docker and environments	Containerize services with separate dev/staging/prod configs. Keep secrets out of images.	Full stack starts through Docker Compose.	Add image build pipeline.
Day 157	Database and migrations	Add migrations, seed data, backup/restore notes, and migration rollback plan.	Run migrations from scratch on a clean DB.	Add test restore from backup.
Day 158	Security hardening	Review auth, RBAC, tenant isolation, secret management, CORS, SSRF, file upload safety, logging redaction, tool sandboxing.	Create a production readiness checklist and fix top issues.	Run dependency audit and secret scan.
Day 159	SLOs and incident thinking	Define SLOs for uptime, p95 latency, job completion, cost anomaly, and answer quality. Write incident playbooks.	Create 3 incident scenarios and response steps.	Add alert rules for one scenario.
Day 160	Deploy	Deploy to a platform you can operate. It can be simple, but it must include environment variables, DB, workers, logs, and a rollback path.	Public/private demo URL works and basic monitoring exists.	Add blue/green or versioned deployment notes.
Day 161	Month 5 review	Review production concepts. Your systems should now have queues, evals, tracing, routing, and deployment.	Month 5 checkpoint: production-shaped AI platform foundation.	Write a readiness gap list before capstone month.

End-of-week evidence

- Runnable artifact for Week 23 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 24: Agent platform architecture: runtime, registry, state, policies, and developer experience

Month 6 - Level 3 foundations and capstone

Goal	Move from building one agent to building an agent platform.
Outcome	A platform blueprint and skeleton implementation for reusable agents, tools, policies, traces, and evals.
Core topics	Agent runtime; Tool registry; Policy layer; State store; Memory store; Eval harness; Developer platform

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 162	Platform blueprint	Design an agent platform with agent definitions, tool registry, MCP adapters, policy engine, state store, memory, queue, tracing, evals, and UI.	Create a platform architecture spec.	Add extensibility points for new agents and tools.
Day 163	Agent definition format	Create a config format for agents: name, purpose, instructions, allowed tools, memory policy, model route, max budget, eval suite.	Load agent definitions from config and run them.	Add validation for unsafe configs.
Day 164	Tool registry v2	Unify internal tools, MCP tools, and API tools behind one registry with schemas, permissions, timeouts, and versions.	One agent can use tools from multiple backends through same gateway.	Add tool discovery UI.
Day 165	State and memory service	Separate workflow state, conversation memory, long-term memory, and audit logs. Define retention and deletion rules.	Implement state tables and memory APIs.	Add memory eval cases to prevent bad memory writes.
Day 166	Policy and guardrails	Apply policy before model call, before tool call, after tool result, before final answer, and before memory write.	Policy blocks at least 5 seeded unsafe scenarios.	Add human override with audit.
Day 167	Developer experience	Create a local dev mode with mock models/tools, trace viewer, replay, eval runner, and sample agent templates.	A new agent can be added in under 30 minutes.	Add CLI command to scaffold an agent.
Day 168	Weekly consolidation	Finalize capstone spec: what you will build, why it matters, success metrics, architecture, risks, and demo script.	Week 24 demo: agent platform skeleton and capstone plan.	Ask for peer review from one strong engineer.

End-of-week evidence

- Runnable artifact for Week 24 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 25: Capstone build: AI engineering agent platform

Month 6 - Level 3 foundations and capstone

Goal	Build a serious portfolio project that proves Level 2 strength and Level 3 direction.
Outcome	A deployed AI engineering agent platform with RAG, agents, MCP, tool gateway, evals, tracing, and policy controls.
Core topics	Capstone implementation; Engineering assistant; MCP integration; RAG + code intelligence; Approvals; Evals; Deployment

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 169	Capstone slice 1: repo intelligence	Implement codebase ingestion, repo Q&A, architecture explanations, and cited file references.	User can ask how a feature works in a repo and receive grounded answer.	Add PR/issue context if available.
Day 170	Capstone slice 2: task planning	Agent converts a bug/feature request into an implementation plan with files, risks, tests, and approval gates.	Agent creates a clear plan for 5 sample engineering tasks.	Add plan critique and revision.
Day 171	Capstone slice 3: safe coding workflow	Read files, propose patch, wait for approval, apply patch, run whitelisted tests, summarize results.	Agent fixes one small bug in a sample repo with trace and rollback.	Add multi-iteration debug loop with max steps.
Day 172	Capstone slice 4: MCP integration	Expose selected capabilities through MCP and consume at least one MCP server through your gateway if safe.	MCP path works without bypassing policy/audit.	Add MCP server trust configuration.
Day 173	Capstone slice 5: evals and traces	Add eval suite for repo Q&A, planning, patch safety, and final answer quality. Add trace views.	Capstone has reproducible eval report and visible traces.	Add comparison between two model routes.
Day 174	Capstone slice 6: production hardening	Review auth, tenant isolation, secrets, logging, queues, caching, cost controls, deployment, and incident docs.	Production readiness checklist is mostly green or has explicit gaps.	Add smoke test after deployment.
Day 175	Weekly consolidation	Create a demo script that shows the platform end to end in 10 minutes.	Week 25 demo: capstone works across core flows.	Record a polished demo video.

End-of-week evidence

- Runnable artifact for Week 25 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

Week 26: Hardening, documentation, portfolio, interview narrative, and next 6 months

Month 6 - Level 3 foundations and capstone

Goal	Convert six months of work into durable skill, credible artifacts, and a next-stage plan toward Level 3.
Outcome	A polished capstone, written architecture case study, demo, eval report, and 6-month continuation roadmap.
Core topics	Hardening; Architecture writing; Portfolio; Interview narrative; Next 6 months; Level 3 roadmap

Day	Focus	2-3 hour work block	Exercise / done condition	4 hour stretch
Day 176	Bug bash	Use your evals, red-team set, and manual testing to identify the top 20 issues. Fix the top 10.	Bug list is prioritized by severity and user impact.	Ask another dev to test it and file issues.
Day 177	Architecture case study	Write a serious case study: problem, architecture, data flow, agent flow, security model, eval strategy, failures, tradeoffs.	Case study is good enough to send to a senior engineer.	Turn diagrams into clean visuals.
Day 178	Portfolio packaging	Clean README, setup guide, screenshots, demo video, eval report, threat model, and roadmap. Remove secrets and private data.	Repo is portfolio-ready or private-demo-ready.	Create a landing page for the project.
Day 179	Level 3 gap review	Assess gaps: distributed systems, deeper Python, Kubernetes, model serving, local inference, compilers/systems, security, eval science.	Create a personalized 6-month next roadmap.	Pick one Level 3 specialization track.
Day 180	Interview and career narrative	Prepare concise explanations for LLM apps, RAG, agents, MCP, evals, observability, security, and your capstone decisions.	Create a 20-question self-interview document.	Practice explaining architecture without buzzwords.
Day 181	Final demo	Run final demo end to end. Show problem -> agent plan -> retrieval -> tool calls -> approval -> patch/test -> trace -> eval report.	Final demo passes without manual database edits or hidden steps.	Record final demo video and preserve trace IDs.
Day 182	Final review and next commitment	Review all notes and projects. Decide the next 6 months: agent platform depth, AI infra, applied product, or model systems.	Final checkpoint: you are Level 2 strong and have Level 3 foundations if you can build, secure, evaluate, and explain this platform.	Publish or share the case study with selected engineers for feedback.

End-of-week evidence

- Runnable artifact for Week 26 committed to Git.
- README or architecture note updated with what changed and why.
- At least one test, eval, or trace proving the feature works.
- Failure notes updated: what broke, what was fixed, what remains risky.

6A. Daily execution cards

Use these cards as the day-level operating system. The weekly table tells you the sequence; the card tells you how to execute without drifting into passive learning. Each card has a learn block, build block, verification block, and 4-hour stretch. Copy the card into your daily notes if you want a working checklist.

Day 1 - Week 1: Orientation and target architecture

Block	Instruction
Learn	Read only what is needed for this task. Topic: Orientation and target architecture. Reference the primary docs for this phase before using courses.
Build	Read the curriculum overview, define your 6-month goal, create a GitHub monorepo, and write a one-page baseline explaining what you currently know about LLM apps, RAG, agents, and MCP.
Verify	Deliver a repo with README, learning log, and a diagram of UI -> API -> LLM -> tools -> data. Commit the baseline honestly.
Stretch	Convert the diagram into a Mermaid or Excalidraw system map.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 2 - Week 1: Environment setup

Block	Instruction
Learn	Read only what is needed for this task. Topic: Environment setup. Reference the primary docs for this phase before using courses.
Build	Install Node LTS, Python 3.12+, uv or poetry, Docker, Postgres, Redis, and a vector DB option. Configure API keys through environment variables only.
Verify	Run a Node health endpoint and a Python health endpoint locally. Add reproducible setup commands to README.
Stretch	Add Docker Compose for Postgres, Redis, and Qdrant.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 3 - Week 1: First LLM request

Block	Instruction
Learn	Read only what is needed for this task. Topic: First LLM request. Reference the primary docs for this phase before using courses.
Build	Use the OpenAI or Anthropic SDK directly. Send a simple request, capture request/response metadata, and print token/cost/latency fields where available.
Verify	Build a /chat endpoint that accepts user input and returns the model answer with latency and trace ID.
Stretch	Add streaming response support from backend to frontend.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 4 - Week 1: Python bridge for MERN developers

Block	Instruction
Learn	Read only what is needed for this task. Topic: Python bridge for MERN developers. Reference the primary docs for this phase before using courses.
Build	Learn Python syntax only where it matters for AI engineering: virtual environments, async, typing, pydantic, FastAPI, file IO, JSON, and HTTP clients.
Verify	Build a FastAPI service with /health, /extract, and /summarize mock endpoints. Call it from Node.
Stretch	Add typed request/response models and input validation.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 5 - Week 1: Minimal chat UI

Block	Instruction
Learn	Read only what is needed for this task. Topic: Minimal chat UI. Reference the primary docs for this phase before using courses.
Build	Build a simple React/Next.js UI for chat, response streaming, model choice, and debug metadata display. Keep it ugly but functional.
Verify	The UI can call your Node endpoint, stream output, and show trace ID, latency, and model used.
Stretch	Persist messages in Postgres with a conversation table.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 6 - Week 1: Logging and repo hygiene

Block	Instruction
Learn	Read only what is needed for this task. Topic: Logging and repo hygiene. Reference the primary docs for this phase before using courses.
Build	Add structured logging, error handling, .env.example, npm scripts, Python scripts, linting, and a simple integration test.
Verify	One command should start the stack. One command should run tests. Document both.
Stretch	Add GitHub Actions for lint and test.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 7 - Week 1: Review and compression

Block	Instruction
Learn	Read only what is needed for this task. Topic: Review and compression. Reference the primary docs for this phase before using courses.
Build	Rebuild the week from memory. Write a short architecture note: what is the LLM doing, what is normal software doing, and where failures can happen.
Verify	Submit Week 1 demo: chat works, logs work, README is usable, and notes identify 5 gaps to fix next week.
Stretch	Record a 5-minute screen walkthrough for yourself.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 8 - Week 2: Prompt anatomy

Block	Instruction
Learn	Read only what is needed for this task. Topic: Prompt anatomy. Reference the primary docs for this phase before using courses.
Build	Study instruction hierarchy, task framing, context placement, output constraints, examples, and refusal/fallback handling. Compare a weak prompt and a production prompt.
Verify	Create a prompt test file with 10 examples: 5 easy, 3 ambiguous, 2 adversarial.
Stretch	Add prompt versioning to your repo.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 9 - Week 2: Structured extraction

Block	Instruction
Learn	Read only what is needed for this task. Topic: Structured extraction. Reference the primary docs for this phase before using courses.
Build	Use schema-based structured output to extract entities from invoices, job posts, or support tickets. Validate output before saving.
Verify	Build /extract-structured that returns validated JSON and rejects invalid outputs.
Stretch	Add retry with a repair prompt only after schema validation fails.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 10 - Week 2: Classification and routing

Block	Instruction
Learn	Read only what is needed for this task. Topic: Classification and routing. Reference the primary docs for this phase before using courses.
Build	Build a classifier for ticket priority, topic, owner, and required action. Keep classes explicit and measurable.
Verify	Create a dataset of 50 synthetic tickets and run batch classification with an accuracy report.
Stretch	Add confidence estimates and manual review threshold.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 11 - Week 2: Streaming UX and cancellation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Streaming UX and cancellation. Reference the primary docs for this phase before using courses.
Build	Implement streaming with cancellation, timeout, and user interruption. Understand how streaming changes frontend state and backend cleanup.
Verify	User can stop generation without corrupting conversation state.
Stretch	Add abort propagation from browser to Node to provider call.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 12 - Week 2: Prompt injection basics

Block	Instruction
Learn	Read only what is needed for this task. Topic: Prompt injection basics. Reference the primary docs for this phase before using courses.
Build	Create examples where user text tries to override instructions. Learn why instruction separation and data labeling matter.
Verify	Add tests proving external data is treated as untrusted content, not instructions.
Stretch	Write a short threat model for your chat endpoint.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 13 - Week 2: Evaluation by examples

Block	Instruction
Learn	Read only what is needed for this task. Topic: Evaluation by examples. Reference the primary docs for this phase before using courses.
Build	Build a tiny eval runner that sends inputs, compares outputs to expected shape/labels, and records pass/fail.
Verify	Run 50 examples and produce a markdown report with failures grouped by category.
Stretch	Add a regression gate in CI for schema and classifier tests.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 14 - Week 2: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Refactor prompts, schemas, tests, and docs. Write down prompt patterns that worked and failed.
Verify	Week 2 demo: structured extraction, classifier, streaming, eval report, and prompt notes.
Stretch	Make a dashboard page that lists eval results.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 15 - Week 3: Tool calling concept

Block	Instruction
Learn	Read only what is needed for this task. Topic: Tool calling concept. Reference the primary docs for this phase before using courses.
Build	Study how tool definitions expose capabilities to a model. Build a mental model: model proposes tool call, app validates, app executes, app returns tool result.
Verify	Write 5 tool specs: get_weather_mock, search_docs_mock, calculate, create_ticket_mock, get_user_profile_mock.
Stretch	Represent tool specs as typed TypeScript objects.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 16 - Week 3: Typed tool registry

Block	Instruction
Learn	Read only what is needed for this task. Topic: Typed tool registry. Reference the primary docs for this phase before using courses.
Build	Build a registry where tools have name, description, input schema, executor, timeout, and permission label.
Verify	The LLM can call calculator and search mock tools through your registry.
Stretch	Add zod validation for every tool input.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 17 - Week 3: Real external API tool

Block	Instruction
Learn	Read only what is needed for this task. Topic: Real external API tool. Reference the primary docs for this phase before using courses.
Build	Connect one safe real API or internal mock API. Handle rate limits, 4xx/5xx errors, and malformed input.
Verify	Create a tool that fetches data and returns a compact result suitable for the model.
Stretch	Add caching for repeated tool calls.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 18 - Week 3: Tool result design

Block	Instruction
Learn	Read only what is needed for this task. Topic: Tool result design. Reference the primary docs for this phase before using courses.
Build	Learn how tool output affects model reasoning. Compare raw JSON, summarized JSON, and pre-filtered results.
Verify	Run 20 test prompts and document which tool-result format creates better answers.
Stretch	Add a max-output-size guard to avoid context bloat.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 19 - Week 3: Approval flow

Block	Instruction
Learn	Read only what is needed for this task. Topic: Approval flow. Reference the primary docs for this phase before using courses.
Build	Add human approval before any tool that writes, deletes, spends money, sends email, or executes code.
Verify	Create a pending_tool_calls table and approval UI.
Stretch	Add replay protection so an approved tool call cannot be executed twice.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 20 - Week 3: Tool observability

Block	Instruction
Learn	Read only what is needed for this task. Topic: Tool observability. Reference the primary docs for this phase before using courses.
Build	Log tool call arguments, duration, status, output size, and user/session IDs. Do not log secrets.
Verify	Produce a tool-call timeline for one conversation.
Stretch	Add a basic trace viewer in the UI.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 21 - Week 3: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Refactor the tool system and write a guide: how to add a new tool, test it, and secure it.
Verify	Week 3 demo: model calls at least 3 tools, approval works, logs are visible, and tests pass.
Stretch	Build the same tool registry in Python to compare ergonomics.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 22 - Week 4: Failure taxonomy

Block	Instruction
Learn	Read only what is needed for this task. Topic: Failure taxonomy. Reference the primary docs for this phase before using courses.
Build	List failures: provider timeout, bad JSON, unsafe request, hallucinated tool use, high latency, excessive cost, rate limit, prompt injection, empty output.
Verify	Create an error taxonomy and map each error to retry, fail, fallback, or human review.
Stretch	Add error codes to API responses.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 23 - Week 4: Retries and fallbacks

Block	Instruction
Learn	Read only what is needed for this task. Topic: Retries and fallbacks. Reference the primary docs for this phase before using courses.
Build	Implement retry only for safe failure modes. Add model fallback for transient provider failures.
Verify	Run tests proving schema failures, rate limits, and timeouts are handled differently.
Stretch	Add circuit breaker behavior after repeated provider failures.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 24 - Week 4: Cost and latency budget

Block	Instruction
Learn	Read only what is needed for this task. Topic: Cost and latency budget. Reference the primary docs for this phase before using courses.
Build	Track tokens, model, duration, and estimated cost per request. Define limits per endpoint.
Verify	Create a cost report for 100 sample calls.
Stretch	Add budget-based routing: cheap model for simple tasks, stronger model for hard tasks.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 25 - Week 4: Guardrails and output validation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Guardrails and output validation. Reference the primary docs for this phase before using courses.
Build	Add input validation, schema validation, refusal handling, output size limits, and unsafe tool protection.
Verify	Every endpoint has a validation layer before model call and before response persistence.
Stretch	Add a simple policy engine for tool permissions.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 26 - Week 4: Mini project build

Block	Instruction
Learn	Read only what is needed for this task. Topic: Mini project build. Reference the primary docs for this phase before using courses.
Build	Build an AI ticket triage assistant: classify ticket, extract fields, propose next action, and optionally create a mock issue after approval.
Verify	End-to-end flow works from UI through API to database and tool registry.
Stretch	Add admin page for failed requests and manual review.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 27 - Week 4: Mini project hardening

Block	Instruction
Learn	Read only what is needed for this task. Topic: Mini project hardening. Reference the primary docs for this phase before using courses.
Build	Add tests, seed data, screenshots, README, architecture diagram, and known limitations.
Verify	The project can be run by another developer using README only.
Stretch	Deploy to a low-cost environment.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 28 - Week 4: Month 1 review

Block	Instruction
Learn	Read only what is needed for this task. Topic: Month 1 review. Reference the primary docs for this phase before using courses.
Build	Review the whole month. Rebuild key pieces from memory: structured output, tool call, streaming, eval runner, approval.
Verify	Month 1 checkpoint: foundation correct if you can explain and demo all primitives without a tutorial.
Stretch	Write a blog-style internal note: LLM features are distributed systems with probabilistic components.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 29 - Week 5: RAG mental model

Block	Instruction
Learn	Read only what is needed for this task. Topic: RAG mental model. Reference the primary docs for this phase before using courses.
Build	Study the RAG pipeline: load, parse, chunk, embed, index, retrieve, rerank, synthesize, cite, evaluate.
Verify	Draw the full RAG architecture and annotate where quality can fail.
Stretch	Compare RAG with fine-tuning in a one-page note.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 30 - Week 5: Embeddings playground

Block	Instruction
Learn	Read only what is needed for this task. Topic: Embeddings playground. Reference the primary docs for this phase before using courses.
Build	Generate embeddings for short texts and inspect nearest neighbors. Learn cosine similarity and why semantic similarity is not truth.
Verify	Create a script that embeds 100 snippets and queries similar snippets.
Stretch	Visualize embeddings in 2D using a simple projection.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 31 - Week 5: Vector DB setup

Block	Instruction
Learn	Read only what is needed for this task. Topic: Vector DB setup. Reference the primary docs for this phase before using courses.
Build	Use pgvector or Qdrant. Create collections/tables with text, embedding, source, metadata, and timestamps.
Verify	Store and query chunks from 10 markdown files.
Stretch	Add metadata filters by source, author, and date.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 32 - Week 5: Chunking experiments

Block	Instruction
Learn	Read only what is needed for this task. Topic: Chunking experiments. Reference the primary docs for this phase before using courses.
Build	Compare fixed-size chunks, heading-aware chunks, and overlap strategies. Measure retrieval quality manually.
Verify	Create a chunking report with examples of good and bad chunks.
Stretch	Implement a pluggable chunker interface.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 33 - Week 5: Basic RAG answerer

Block	Instruction
Learn	Read only what is needed for this task. Topic: Basic RAG answerer. Reference the primary docs for this phase before using courses.
Build	Retrieve top chunks, build a context block, ask the model to answer only from context, and return citations.
Verify	Build /ask-docs endpoint with answer, cited chunks, and no-answer behavior.
Stretch	Add streaming answer with cited sources shown before final answer.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 34 - Week 5: Retrieval tests

Block	Instruction
Learn	Read only what is needed for this task. Topic: Retrieval tests. Reference the primary docs for this phase before using courses.
Build	Create 30 question-answer pairs from your documents. Track whether retrieval found the correct chunk.
Verify	Produce retrieval@k measurements manually or with a simple script.
Stretch	Add failed-query analysis to learning notes.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 35 - Week 5: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Refactor ingestion and retrieval. Document chunking decisions and known retrieval failure modes.
Verify	Week 5 demo: upload docs, index docs, ask questions, receive cited answers.
Stretch	Swap vector DB and compare implementation effort.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 36 - Week 6: Ingestion architecture

Block	Instruction
Learn	Read only what is needed for this task. Topic: Ingestion architecture. Reference the primary docs for this phase before using courses.
Build	Design loader -> parser -> cleaner -> chunker -> embedder -> indexer -> verifier. Keep each stage testable.
Verify	Create pipeline interfaces and a CLI command to ingest a folder.
Stretch	Add a job table for ingestion status.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 37 - Week 6: Markdown and text ingestion

Block	Instruction
Learn	Read only what is needed for this task. Topic: Markdown and text ingestion. Reference the primary docs for this phase before using courses.
Build	Implement stable ingestion for markdown, txt, and JSON. Preserve headings and source paths.
Verify	Index 50 files and confirm metadata is searchable.
Stretch	Add content hash dedupe.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 38 - Week 6: PDF ingestion

Block	Instruction
Learn	Read only what is needed for this task. Topic: PDF ingestion. Reference the primary docs for this phase before using courses.
Build	Parse PDFs carefully and capture page numbers. Identify where extraction fails and where OCR would be needed.
Verify	Index 3 PDFs with page-level citations.
Stretch	Add a parser quality score per document.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 39 - Week 6: Web page ingestion

Block	Instruction
Learn	Read only what is needed for this task. Topic: Web page ingestion. Reference the primary docs for this phase before using courses.
Build	Fetch public pages, clean boilerplate, respect source URL, and chunk by headings.
Verify	Index 10 web pages and answer questions with URL citations.
Stretch	Add robots/respectful-fetch notes to README.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 40 - Week 6: Codebase ingestion

Block	Instruction
Learn	Read only what is needed for this task. Topic: Codebase ingestion. Reference the primary docs for this phase before using courses.
Build	Chunk code by file, function/class boundaries where possible, and keep path, language, exports, imports, and repo metadata.
Verify	Index one MERN repo and ask codebase questions.
Stretch	Add separate retrieval modes for docs and code.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 41 - Week 6: Incremental reindexing

Block	Instruction
Learn	Read only what is needed for this task. Topic: Incremental reindexing. Reference the primary docs for this phase before using courses.
Build	Detect changed files, deleted files, and unchanged files. Avoid re-embedding everything.
Verify	Run ingestion twice and prove unchanged files are skipped.
Stretch	Add reindex queue with retry.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 42 - Week 6: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Write a pipeline design doc and run ingestion against a realistic mixed corpus.
Verify	Week 6 demo: folder ingestion, PDF ingestion, code ingestion, incremental reindexing.
Stretch	Add admin UI for ingestion jobs.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 43 - Week 7: Why naive RAG fails

Block	Instruction
Learn	Read only what is needed for this task. Topic: Why naive RAG fails. Reference the primary docs for this phase before using courses.
Build	Study failure modes: wrong chunk, missing chunk, too many chunks, ambiguous query, stale source, permission mismatch, and poor synthesis.
Verify	Write 20 failure examples against your existing system.
Stretch	Categorize failures as ingestion, retrieval, ranking, prompt, or data issue.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 44 - Week 7: Keyword + vector retrieval

Block	Instruction
Learn	Read only what is needed for this task. Topic: Keyword + vector retrieval. Reference the primary docs for this phase before using courses.
Build	Add keyword/BM25 or database full-text search alongside vector search.
Verify	Compare vector-only vs keyword-only vs hybrid on 30 questions.
Stretch	Implement reciprocal-rank-fusion style merging.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 45 - Week 7: Reranking

Block	Instruction
Learn	Read only what is needed for this task. Topic: Reranking. Reference the primary docs for this phase before using courses.
Build	Add a rerank step using a reranker model or provider capability. Keep input size controlled.
Verify	Measure whether reranking improves top-3 relevance.
Stretch	Add rerank cost and latency tracking.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 46 - Week 7: Query transformation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Query transformation. Reference the primary docs for this phase before using courses.
Build	Implement query rewrite, multi-query generation, and acronym expansion. Do not blindly trust generated queries.
Verify	Run query transformation on ambiguous codebase questions.
Stretch	Add a toggle to compare raw vs rewritten query.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 47 - Week 7: Metadata filters and permissions

Block	Instruction
Learn	Read only what is needed for this task. Topic: Metadata filters and permissions. Reference the primary docs for this phase before using courses.
Build	Add filters by tenant, repo, file type, date, owner, and permission group.
Verify	Prove a user cannot retrieve chunks outside their permission boundary.
Stretch	Add test fixtures for cross-tenant leakage.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 48 - Week 7: Answer synthesis

Block	Instruction
Learn	Read only what is needed for this task. Topic: Answer synthesis. Reference the primary docs for this phase before using courses.
Build	Improve prompt design: answer from context, cite each claim, say insufficient evidence when needed, avoid over-answering.
Verify	Run an answer quality eval over 30 questions.
Stretch	Add citation coverage score.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 49 - Week 7: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Create a retrieval quality dashboard with failure categories, top-k hit rate, latency, and cost.
Verify	Week 7 demo: hybrid retrieval, reranking, query rewriting, metadata filters.
Stretch	Write a deep note: retrieval is the real product, the model is the final formatter.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 50 - Week 8: Golden dataset design

Block	Instruction
Learn	Read only what is needed for this task. Topic: Golden dataset design. Reference the primary docs for this phase before using courses.
Build	Create a small but serious evaluation set: exact-answer questions, summary questions, comparison questions, and unanswerable questions.
Verify	Build a dataset with 80 questions and expected evidence IDs.
Stretch	Add a JSONL format for eval cases.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 51 - Week 8: Retrieval evaluation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Retrieval evaluation. Reference the primary docs for this phase before using courses.
Build	Measure if correct evidence appears in top-k. Separate retrieval failure from generation failure.
Verify	Produce retrieval hit rate at k=3, k=5, and k=10.
Stretch	Add per-source and per-file-type breakdown.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 52 - Week 8: Answer evaluation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Answer evaluation. Reference the primary docs for this phase before using courses.
Build	Evaluate groundedness, correctness, completeness, and citation coverage. Use human review first, optional LLM judge later.
Verify	Review 30 answers and record failure reasons.
Stretch	Add an LLM-as-judge experiment with human spot check.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 53 - Week 8: Repo-aware assistant

Block	Instruction
Learn	Read only what is needed for this task. Topic: Repo-aware assistant. Reference the primary docs for this phase before using courses.
Build	Index a real MERN repo. Build features: explain file, trace API flow, find auth logic, answer where a bug should be fixed.
Verify	Demo 10 codebase questions with cited files and line/page-like references where possible.
Stretch	Add code search mode separated from docs search mode.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 54 - Week 8: Hallucination controls

Block	Instruction
Learn	Read only what is needed for this task. Topic: Hallucination controls. Reference the primary docs for this phase before using courses.
Build	Add refusal behavior, answer confidence based on retrieval quality, source requirement, and no-answer path.
Verify	System refuses 10 intentionally unanswerable questions.
Stretch	Add source-aware response templates for explain, compare, and summarize.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 55 - Week 8: Regression suite

Block	Instruction
Learn	Read only what is needed for this task. Topic: Regression suite. Reference the primary docs for this phase before using courses.
Build	Run the full eval suite before and after prompt/retrieval changes. Track regressions.
Verify	Create a report comparing two versions of your RAG pipeline.
Stretch	Add CI job for a small smoke eval.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 56 - Week 8: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Package the repo assistant as a portfolio project with architecture diagram, eval results, and failure analysis.
Verify	Week 8 demo: repo assistant with eval report and grounded answers.
Stretch	Record a demo where you use it to onboard into an unfamiliar repo.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 57 - Week 9: Multi-tenant design

Block	Instruction
Learn	Read only what is needed for this task. Topic: Multi-tenant design. Reference the primary docs for this phase before using courses.
Build	Design tenant, workspace, user, source, document, chunk, embedding, job, and query tables. Identify permission checks.
Verify	Write schema migrations or ERD for multi-tenant RAG.
Stretch	Add row-level security or app-level permission tests.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 58 - Week 9: Freshness and reindexing

Block	Instruction
Learn	Read only what is needed for this task. Topic: Freshness and reindexing. Reference the primary docs for this phase before using courses.
Build	Track document modified time, hash, index version, embedding model version, and stale chunks.
Verify	Implement reindex for changed sources and cleanup for deleted documents.
Stretch	Add index versioning to allow rollback.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 59 - Week 9: Caching

Block	Instruction
Learn	Read only what is needed for this task. Topic: Caching. Reference the primary docs for this phase before using courses.
Build	Cache embeddings, retrieval results, model responses where safe, and rerank outputs. Know what must not be cached.
Verify	Add cache hit/miss metrics and invalidation rules.
Stretch	Add semantic cache experiment for repeated questions.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 60 - Week 9: Summarization layer

Block	Instruction
Learn	Read only what is needed for this task. Topic: Summarization layer. Reference the primary docs for this phase before using courses.
Build	Build document summaries, folder/source summaries, and conversation summaries. Understand summary drift.
Verify	Create summaries with source links and timestamps.
Stretch	Add a summary refresh job when underlying docs change.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 61 - Week 9: Observability for RAG

Block	Instruction
Learn	Read only what is needed for this task. Topic: Observability for RAG. Reference the primary docs for this phase before using courses.
Build	Log query, rewritten query, retrieved chunk IDs, scores, rerank scores, answer, citations, cost, latency, and user feedback.
Verify	Create a query trace view for one answer.
Stretch	Add thumbs-up/down feedback tied to trace ID.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 62 - Week 9: Month 2 hardening

Block	Instruction
Learn	Read only what is needed for this task. Topic: Month 2 hardening. Reference the primary docs for this phase before using courses.
Build	Run security checks for permissions, prompt injection through retrieved docs, oversized chunks, and stale sources.
Verify	Show tests for cross-tenant leakage and injected document text.
Stretch	Add a red-team dataset of malicious documents.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 63 - Week 9: Month 2 review

Block	Instruction
Learn	Read only what is needed for this task. Topic: Month 2 review. Reference the primary docs for this phase before using courses.
Build	Consolidate all RAG learnings into one architecture document. Compare your system against naive RAG.
Verify	Month 2 checkpoint: production-shaped repo/document assistant with evals and security tests.
Stretch	Write a case study: RAG quality comes from data and retrieval engineering.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 64 - Week 10: Agent definition

Block	Instruction
Learn	Read only what is needed for this task. Topic: Agent definition. Reference the primary docs for this phase before using courses.
Build	Define agent precisely: model plus instructions plus state plus tools plus loop plus stopping rules plus memory plus policy.
Verify	Write an agent runtime design doc with sequence diagram.
Stretch	Compare chatbot, workflow, and agent in a table.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 65 - Week 10: Build the loop

Block	Instruction
Learn	Read only what is needed for this task. Topic: Build the loop. Reference the primary docs for this phase before using courses.
Build	Implement a loop: send state to model, receive message or tool call, validate tool call, execute tool, append result, repeat until final or max steps.
Verify	Agent can use calculator and search mock tools over multiple steps.
Stretch	Add step-by-step trace display.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 66 - Week 10: State and memory

Block	Instruction
Learn	Read only what is needed for this task. Topic: State and memory. Reference the primary docs for this phase before using courses.
Build	Separate short-term state, conversation memory, long-term memory, and external knowledge. Do not dump everything into context.
Verify	Add conversation summary memory and user preference memory.
Stretch	Add memory write approval to prevent accidental persistence.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 67 - Week 10: Planning strategies

Block	Instruction
Learn	Read only what is needed for this task. Topic: Planning strategies. Reference the primary docs for this phase before using courses.
Build	Compare no-plan, plan-first, plan-and-execute, and ReAct-style loops. Keep plans inspectable.
Verify	Run the same 10 tasks with two planning modes and compare failures.
Stretch	Add plan revision after tool failure.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 68 - Week 10: Safety limits

Block	Instruction
Learn	Read only what is needed for this task. Topic: Safety limits. Reference the primary docs for this phase before using courses.
Build	Add max steps, max tool calls, max cost, max execution time, tool allowlist, and dangerous action approval.
Verify	Demonstrate a task that stops safely instead of looping forever.
Stretch	Add loop-detection based on repeated tool calls.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 69 - Week 10: Agent task: research assistant

Block	Instruction
Learn	Read only what is needed for this task. Topic: Agent task: research assistant. Reference the primary docs for this phase before using courses.
Build	Build a simple research assistant that searches docs/web mock, extracts evidence, drafts answer, and cites sources.
Verify	Agent completes 5 research tasks with trace and citations.
Stretch	Add critique pass before final answer.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 70 - Week 10: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Write down every failure mode observed: bad plan, bad tool input, wrong memory, looping, weak evidence, overconfident answer.
Verify	Week 10 demo: custom agent runtime with trace, tools, memory, limits.
Stretch	Refactor runtime into reusable package.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 71 - Week 11: LangGraph basics

Block	Instruction
Learn	Read only what is needed for this task. Topic: LangGraph basics. Reference the primary docs for this phase before using courses.
Build	Study graph concepts: node, edge, state, reducer, conditional route, checkpoint. Map your custom loop into graph terms.
Verify	Build a two-node graph: classify request -> answer.
Stretch	Add graph visualization if possible.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 72 - Week 11: Tool-using graph

Block	Instruction
Learn	Read only what is needed for this task. Topic: Tool-using graph. Reference the primary docs for this phase before using courses.
Build	Create nodes for plan, retrieve, tool_execute, synthesize, and final. Keep state explicit.
Verify	Run a RAG agent workflow through graph nodes.
Stretch	Add node-level timing and errors.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 73 - Week 11: Conditional routes

Block	Instruction
Learn	Read only what is needed for this task. Topic: Conditional routes. Reference the primary docs for this phase before using courses.
Build	Route based on confidence, tool need, retrieval score, and approval requirement.
Verify	Low-confidence queries go to clarification or refusal instead of answer.
Stretch	Add branch coverage tests.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 74 - Week 11: Persistence and resume

Block	Instruction
Learn	Read only what is needed for this task. Topic: Persistence and resume. Reference the primary docs for this phase before using courses.
Build	Use checkpoints or your own persistence to resume an interrupted workflow.
Verify	Start a workflow, interrupt it, resume from saved state.
Stretch	Add resumable state to the UI.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 75 - Week 11: Human-in-the-loop

Block	Instruction
Learn	Read only what is needed for this task. Topic: Human-in-the-loop. Reference the primary docs for this phase before using courses.
Build	Pause before risky tool execution or final external action. Allow approve, reject, or edit.
Verify	Workflow pauses for approval and resumes after decision.
Stretch	Add audit records for decisions.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 76 - Week 11: Streaming progress

Block	Instruction
Learn	Read only what is needed for this task. Topic: Streaming progress. Reference the primary docs for this phase before using courses.
Build	Stream graph progress to UI: current node, events, tool calls, final answer.
Verify	User sees live agent progress without exposing secrets.
Stretch	Add collapsible trace details.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 77 - Week 11: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Build a durable ticket-resolution agent using LangGraph: classify, retrieve policy, draft answer, ask approval, create ticket.
Verify	Week 11 demo: stateful workflow with persistence and approval.
Stretch	Write a note comparing custom loop vs LangGraph.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 78 - Week 12: Research workflow design

Block	Instruction
Learn	Read only what is needed for this task. Topic: Research workflow design. Reference the primary docs for this phase before using courses.
Build	Design research as stages: understand question, generate search plan, gather sources, extract claims, verify, synthesize, cite.
Verify	Write a research-agent system design with failure modes.
Stretch	Add source quality criteria to the design.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 79 - Week 12: Search and fetch tools

Block	Instruction
Learn	Read only what is needed for this task. Topic: Search and fetch tools. Reference the primary docs for this phase before using courses.
Build	Build tools for search/fetch or use mock sources if needed. Clean pages and preserve source metadata.
Verify	Agent can collect source snippets and store them as evidence objects.
Stretch	Add source deduplication.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 80 - Week 12: Evidence extraction

Block	Instruction
Learn	Read only what is needed for this task. Topic: Evidence extraction. Reference the primary docs for this phase before using courses.
Build	Extract atomic claims from sources with quote/source/url/date fields. Avoid long copyrighted quotes in outputs.
Verify	Build an evidence table for 5 research tasks.
Stretch	Add claim type: fact, opinion, instruction, code, date-sensitive.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 81 - Week 12: Cross-checking

Block	Instruction
Learn	Read only what is needed for this task. Topic: Cross-checking. Reference the primary docs for this phase before using courses.
Build	Require important claims to be supported by multiple sources when possible. Mark conflicts clearly.
Verify	Agent flags conflicting evidence instead of choosing silently.
Stretch	Add a contradiction detector prompt or rule.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 82 - Week 12: Synthesis and citations

Block	Instruction
Learn	Read only what is needed for this task. Topic: Synthesis and citations. Reference the primary docs for this phase before using courses.
Build	Generate answers from evidence objects, not raw pages. Each major claim should map to source IDs.
Verify	Final answer includes citations and uncertainty where evidence is weak.
Stretch	Add answer sections: findings, evidence, unknowns.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 83 - Week 12: Evaluation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Evaluation. Reference the primary docs for this phase before using courses.
Build	Create 20 research tasks and grade evidence quality, answer correctness, citation coverage, and overclaiming.
Verify	Produce a research-agent eval report.
Stretch	Add a verification pass that critiques final answer before user sees it.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 84 - Week 12: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Package the research agent with trace UI and evidence table.
Verify	Week 12 demo: research agent with sources, verification, and clear unknowns.
Stretch	Use it to research one AI engineering topic you will study next.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 85 - Week 13: Coding-agent architecture

Block	Instruction
Learn	Read only what is needed for this task. Topic: Coding-agent architecture. Reference the primary docs for this phase before using courses.
Build	Study the loop: inspect repo, choose files, propose changes, apply patch, run tests, inspect failure, iterate.
Verify	Write a coding-agent threat model before coding anything.
Stretch	List tools by risk level: read, write, execute, network.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 86 - Week 13: Read-only repo assistant

Block	Instruction
Learn	Read only what is needed for this task. Topic: Read-only repo assistant. Reference the primary docs for this phase before using courses.
Build	Build tools: list_files, read_file, search_files, grep, get_package_scripts. Keep path traversal blocked.
Verify	Agent answers repo questions using read-only tools.
Stretch	Add max file size and allowed directory restrictions.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 87 - Week 13: Patch proposal

Block	Instruction
Learn	Read only what is needed for this task. Topic: Patch proposal. Reference the primary docs for this phase before using courses.
Build	Ask the model to produce a patch plan and unified diff. Do not auto-apply yet.
Verify	Agent proposes patches for 3 simple bugs and explains expected tests.
Stretch	Add diff validation.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 88 - Week 13: Apply patch with approval

Block	Instruction
Learn	Read only what is needed for this task. Topic: Apply patch with approval. Reference the primary docs for this phase before using courses.
Build	Implement apply_patch only after approval. Store before/after diff and allow rollback.
Verify	Patch can be approved, applied, and reverted.
Stretch	Add a UI diff viewer.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 89 - Week 13: Command execution sandbox

Block	Instruction
Learn	Read only what is needed for this task. Topic: Command execution sandbox. Reference the primary docs for this phase before using courses.
Build	Run safe test commands in a constrained environment. Block arbitrary shell by default.
Verify	Agent can run npm test or a whitelisted command and read output.
Stretch	Add timeouts and output truncation.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 90 - Week 13: Debug loop

Block	Instruction
Learn	Read only what is needed for this task. Topic: Debug loop. Reference the primary docs for this phase before using courses.
Build	Let the agent inspect test failure and propose one fix iteration. Keep max iterations low.
Verify	Agent fixes a deliberately broken small project.
Stretch	Add a benchmark of 10 small bugs.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 91 - Week 13: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Document limitations: why coding agents need sandboxing, approvals, and test grounding.
Verify	Week 13 demo: read-only analysis, approved patch, test run, rollback.
Stretch	Compare your flow with Cursor/Claude Code at a conceptual level.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 92 - Week 14: When multi-agent is useful

Block	Instruction
Learn	Read only what is needed for this task. Topic: When multi-agent is useful. Reference the primary docs for this phase before using courses.
Build	Study patterns: supervisor, router, specialist, critic, planner/executor, agents-as-tools. Also study when a normal workflow is better.
Verify	Write decision rules for single workflow vs single agent vs multi-agent.
Stretch	Draw architecture for 3 agent patterns.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 93 - Week 14: Supervisor + specialist

Block	Instruction
Learn	Read only what is needed for this task. Topic: Supervisor + specialist. Reference the primary docs for this phase before using courses.
Build	Build a supervisor that routes to RAG specialist, coding specialist, or research specialist. Keep tool permissions separate.
Verify	Supervisor routes 15 tasks correctly.
Stretch	Add fallback when specialist confidence is low.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 94 - Week 14: Critic/reviewer agent

Block	Instruction
Learn	Read only what is needed for this task. Topic: Critic/reviewer agent. Reference the primary docs for this phase before using courses.
Build	Add a reviewer that checks grounding, security, completeness, and format before final response.
Verify	Reviewer catches at least 5 seeded errors.
Stretch	Compare self-critique vs separate reviewer.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 95 - Week 14: Handoffs and state

Block	Instruction
Learn	Read only what is needed for this task. Topic: Handoffs and state. Reference the primary docs for this phase before using courses.
Build	Implement handoff with state summary, task goal, constraints, and expected output. Avoid dumping full conversation blindly.
Verify	Handoff logs are inspectable and small.
Stretch	Add handoff schema validation.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 96 - Week 14: Multi-agent evals

Block	Instruction
Learn	Read only what is needed for this task. Topic: Multi-agent evals. Reference the primary docs for this phase before using courses.
Build	Create an eval set for routing correctness, final answer quality, cost, and latency.
Verify	Measure if multi-agent improves quality enough to justify cost.
Stretch	Add a no-agent baseline comparison.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 97 - Week 14: Month 3 integrated project

Block	Instruction
Learn	Read only what is needed for this task. Topic: Month 3 integrated project. Reference the primary docs for this phase before using courses.
Build	Build an AI engineering assistant: can answer repo questions, research docs, propose patch plan, and ask approval for risky actions.
Verify	Integrated demo uses RAG, tools, agent loop, graph workflow, and approval.
Stretch	Deploy a private demo environment.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 98 - Week 14: Month 3 review

Block	Instruction
Learn	Read only what is needed for this task. Topic: Month 3 review. Reference the primary docs for this phase before using courses.
Build	Review foundations. You should now understand LLM primitives, RAG, agents, workflows, tools, state, memory, and evaluation.
Verify	Month 3 checkpoint: you can build AI agents from scratch and with LangGraph, not just use tools.
Stretch	Write your Level 2 readiness assessment and gaps.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 99 - Week 15: MCP mental model

Block	Instruction
Learn	Read only what is needed for this task. Topic: MCP mental model. Reference the primary docs for this phase before using courses.
Build	Study MCP client, server, tools, resources, prompts, transport, and capability discovery. Compare MCP with REST and plugin systems.
Verify	Write a one-page explanation of MCP for a MERN developer.
Stretch	Map your Week 3 tool registry to MCP terms.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 100 - Week 15: First MCP server

Block	Instruction
Learn	Read only what is needed for this task. Topic: First MCP server. Reference the primary docs for this phase before using courses.
Build	Build a minimal MCP server in TypeScript or Python exposing calculator and echo-safe tools.
Verify	Connect it to a compatible MCP client and call the tools.
Stretch	Add structured input schemas for all tools.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 101 - Week 15: Resources and prompts

Block	Instruction
Learn	Read only what is needed for this task. Topic: Resources and prompts. Reference the primary docs for this phase before using courses.
Build	Expose read-only resources such as project README, package metadata, or docs snippets. Add prompt templates if supported.
Verify	Client can list resources and use them as context.
Stretch	Add resource pagination or size limits.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 102 - Week 15: Transport and lifecycle

Block	Instruction
Learn	Read only what is needed for this task. Topic: Transport and lifecycle. Reference the primary docs for this phase before using courses.
Build	Understand STDIO vs HTTP/SSE or streamable HTTP options as applicable. Learn startup, shutdown, logging, and error behavior.
Verify	Document how your server starts and how failures appear in the client.
Stretch	Add health check and version endpoint where relevant.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 103 - Week 15: MCP server from existing tools

Block	Instruction
Learn	Read only what is needed for this task. Topic: MCP server from existing tools. Reference the primary docs for this phase before using courses.
Build	Wrap your existing tool registry into MCP tools. Keep permission labels and validation.
Verify	At least 3 existing tools are available through MCP.
Stretch	Create a small adapter layer between internal tools and MCP schema.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 104 - Week 15: Testing MCP tools

Block	Instruction
Learn	Read only what is needed for this task. Topic: Testing MCP tools. Reference the primary docs for this phase before using courses.
Build	Write tests for tool schemas, invalid inputs, permission failures, and tool output size.
Verify	MCP server test suite passes locally.
Stretch	Add mock client tests.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 105 - Week 15: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Write setup instructions and a diagram: AI client -> MCP server -> tool gateway -> data/API.
Verify	Week 15 demo: functional MCP server with tools/resources and tests.
Stretch	Connect the server to both a coding client and desktop client if available.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 106 - Week 16: Tool boundary design

Block	Instruction
Learn	Read only what is needed for this task. Topic: Tool boundary design. Reference the primary docs for this phase before using courses.
Build	Classify tools: read-only, write with approval, destructive, external side effect, code execution. Design separate namespaces.
Verify	Create a tool risk matrix for your MCP server.
Stretch	Add risk labels to tool descriptions.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 107 - Week 16: Database read tools

Block	Instruction
Learn	Read only what is needed for this task. Topic: Database read tools. Reference the primary docs for this phase before using courses.
Build	Expose safe DB read tools with parameterized queries only. Never allow raw SQL from model/user.
Verify	Tool can fetch ticket by ID and search tickets by filters.
Stretch	Add tenant filter enforcement.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 108 - Week 16: GitHub/repo tools

Block	Instruction
Learn	Read only what is needed for this task. Topic: GitHub/repo tools. Reference the primary docs for this phase before using courses.
Build	Expose read-only repo tools: list issues mock/live, read file, search code, inspect PR metadata.
Verify	Client can ask repo questions through MCP tools.
Stretch	Add rate-limit handling.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 109 - Week 16: Filesystem safety

Block	Instruction
Learn	Read only what is needed for this task. Topic: Filesystem safety. Reference the primary docs for this phase before using courses.
Build	Implement safe filesystem access with root jail, path normalization, file size limits, and read-only default.
Verify	Tests prove path traversal does not work.
Stretch	Add allowlist by file extension.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 110 - Week 16: Approved write action

Block	Instruction
Learn	Read only what is needed for this task. Topic: Approved write action. Reference the primary docs for this phase before using courses.
Build	Create one write action such as create_mock_issue or draft_pr_description. Require approval outside the model.
Verify	Write action cannot execute without explicit approval token.
Stretch	Add audit log entry with actor, args, and result.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 111 - Week 16: Tool descriptions and UX

Block	Instruction
Learn	Read only what is needed for this task. Topic: Tool descriptions and UX. Reference the primary docs for this phase before using courses.
Build	Improve tool names/descriptions so models choose correctly. Remove vague tools.
Verify	Run 30 prompts and measure correct tool selection.
Stretch	Add a bad-tool-selection test set.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 112 - Week 16: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Package tool suite with docs and example prompts. Review all tools for least privilege.
Verify	Week 16 demo: DB, repo, filesystem, and approved write tools exposed via MCP.
Stretch	Create a public-safe demo variant with mock data.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 113 - Week 17: AI security baseline

Block	Instruction
Learn	Read only what is needed for this task. Topic: AI security baseline. Reference the primary docs for this phase before using courses.
Build	Study OWASP LLM and Agentic AI risks. Translate each risk into developer controls for your stack.
Verify	Create a security checklist for every AI feature you build.
Stretch	Map controls to code locations in your repo.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 114 - Week 17: Prompt injection red-team

Block	Instruction
Learn	Read only what is needed for this task. Topic: Prompt injection red-team. Reference the primary docs for this phase before using courses.
Build	Create malicious documents, malicious tool outputs, and malicious user prompts. Test if the agent follows them.
Verify	Add red-team tests that your agent must fail safely.
Stretch	Add instruction/data labeling to all retrieved context.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 115 - Week 17: Tool poisoning and rug-pull risks

Block	Instruction
Learn	Read only what is needed for this task. Topic: Tool poisoning and rug-pull risks. Reference the primary docs for this phase before using courses.
Build	Understand risks from tool descriptions, changing tool behavior, untrusted MCP servers, and marketplace-style integrations.
Verify	Add tool versioning, approval, and pinned trusted tool config.
Stretch	Hash tool definitions and alert on changes.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 116 - Week 17: Sandboxing execution

Block	Instruction
Learn	Read only what is needed for this task. Topic: Sandboxing execution. Reference the primary docs for this phase before using courses.
Build	Isolate any command/code execution. Restrict network, filesystem, environment variables, time, memory, and allowed commands.
Verify	Dangerous command requests are blocked and logged.
Stretch	Run test commands in a disposable container.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 117 - Week 17: Secrets and data protection

Block	Instruction
Learn	Read only what is needed for this task. Topic: Secrets and data protection. Reference the primary docs for this phase before using courses.
Build	Prevent model access to secrets. Scrub logs. Avoid sending sensitive data unnecessarily to model providers.
Verify	Add secret scanning and log redaction tests.
Stretch	Add data classification labels to tool outputs.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 118 - Week 17: Authorization and approval

Block	Instruction
Learn	Read only what is needed for this task. Topic: Authorization and approval. Reference the primary docs for this phase before using courses.
Build	Design auth for human users, agents, and tools. Enforce user permissions before tool execution.
Verify	Write tests for unauthorized tool calls and cross-tenant access.
Stretch	Add approval UI with reason and diff preview.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 119 - Week 17: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Produce a threat model for your MCP + agent platform: assets, actors, trust boundaries, attacks, controls, residual risk.
Verify	Week 17 demo: red-team tests, sandbox, approvals, secret redaction, threat model.
Stretch	Ask another developer to review your threat model.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 120 - Week 18: Tool gateway design

Block	Instruction
Learn	Read only what is needed for this task. Topic: Tool gateway design. Reference the primary docs for this phase before using courses.
Build	Design a gateway that all agent tools go through: validate, authorize, approve, execute, log, return.
Verify	Create gateway sequence diagram and API contract.
Stretch	Add policy decision point and policy enforcement point vocabulary.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 121 - Week 18: Policy engine

Block	Instruction
Learn	Read only what is needed for this task. Topic: Policy engine. Reference the primary docs for this phase before using courses.
Build	Implement simple policies: user role, tenant, tool risk, environment, time budget, data classification, approval requirement.
Verify	Policy denies risky calls automatically and explains why.
Stretch	Use a declarative policy config file.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 122 - Week 18: Audit logging

Block	Instruction
Learn	Read only what is needed for this task. Topic: Audit logging. Reference the primary docs for this phase before using courses.
Build	Record actor, user, tenant, tool, args hash, approval, result summary, duration, cost, and trace ID.
Verify	Audit log can reconstruct an agent action timeline.
Stretch	Add immutable append-only log table pattern.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 123 - Week 18: Queued execution

Block	Instruction
Learn	Read only what is needed for this task. Topic: Queued execution. Reference the primary docs for this phase before using courses.
Build	Move slow/risky tool execution to a job queue. Support pending, running, succeeded, failed, cancelled.
Verify	Agent receives job status and final result through polling or event stream.
Stretch	Add retry policies per tool.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 124 - Week 18: Runtime isolation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Runtime isolation. Reference the primary docs for this phase before using courses.
Build	Separate model orchestration, tool gateway, data services, and UI. Avoid one giant backend.
Verify	Refactor services or modules to clear boundaries.
Stretch	Add service-level README for each boundary.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 125 - Week 18: Integrated MCP gateway

Block	Instruction
Learn	Read only what is needed for this task. Topic: Integrated MCP gateway. Reference the primary docs for this phase before using courses.
Build	Route MCP tool calls through your gateway. Do not let MCP handlers directly execute dangerous actions.
Verify	MCP tools now inherit auth, policy, audit, and approval.
Stretch	Add smoke tests from MCP client through gateway.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 126 - Week 18: Month 4 review

Block	Instruction
Learn	Read only what is needed for this task. Topic: Month 4 review. Reference the primary docs for this phase before using courses.
Build	Review MCP, security, gateway, and infra concepts. Update your portfolio diagrams.
Verify	Month 4 checkpoint: you can build and secure MCP servers, not just connect random ones.
Stretch	Write a paper-style note: safe agent systems need infrastructure, not vibes.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 127 - Week 19: Async architecture

Block	Instruction
Learn	Read only what is needed for this task. Topic: Async architecture. Reference the primary docs for this phase before using courses.
Build	Design which work stays synchronous and which becomes queued: indexing, evals, long agent tasks, batch extraction, report generation.
Verify	Create a queue architecture diagram.
Stretch	Choose BullMQ, Celery, Temporal, or equivalent for experiments.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 128 - Week 19: Basic worker

Block	Instruction
Learn	Read only what is needed for this task. Topic: Basic worker. Reference the primary docs for this phase before using courses.
Build	Implement queue + worker for document ingestion or long extraction. Track job status in DB.
Verify	User can submit job and view status.
Stretch	Add progress events.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 129 - Week 19: Retries and idempotency

Block	Instruction
Learn	Read only what is needed for this task. Topic: Retries and idempotency. Reference the primary docs for this phase before using courses.
Build	Add retry rules, idempotency keys, dedupe, and safe resume. Understand why retries can duplicate side effects.
Verify	Prove repeated job submission does not duplicate documents/actions.
Stretch	Add exactly-once-like behavior where feasible.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 130 - Week 19: Cancellation and timeouts

Block	Instruction
Learn	Read only what is needed for this task. Topic: Cancellation and timeouts. Reference the primary docs for this phase before using courses.
Build	Support cancellation for long model calls, tool calls, and workflows. Persist cancelled state.
Verify	User can cancel a running job and see partial trace.
Stretch	Add cleanup logic for temporary files.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 131 - Week 19: Long-running agent workflow

Block	Instruction
Learn	Read only what is needed for this task. Topic: Long-running agent workflow. Reference the primary docs for this phase before using courses.
Build	Run an agent workflow through the queue. Store each step and allow resume after crash.
Verify	Agent survives process restart or can resume from last checkpoint.
Stretch	Add watchdog for stuck jobs.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 132 - Week 19: Backpressure and rate limits

Block	Instruction
Learn	Read only what is needed for this task. Topic: Backpressure and rate limits. Reference the primary docs for this phase before using courses.
Build	Control concurrency by tenant, tool, model, and queue. Prevent runaway agents.
Verify	Load test with many jobs and show queue metrics.
Stretch	Add per-tenant quotas.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 133 - Week 19: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Document worker architecture, retries, idempotency, and failure handling.
Verify	Week 19 demo: queued agent task with progress, cancellation, retry, and resume.
Stretch	Compare queue-based orchestration with graph checkpointing.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 134 - Week 20: Observability model

Block	Instruction
Learn	Read only what is needed for this task. Topic: Observability model. Reference the primary docs for this phase before using courses.
Build	List what to observe: prompt version, model, input size, output size, cost, latency, retrieval IDs, tool calls, errors, eval scores, feedback.
Verify	Create an observability schema for your AI platform.
Stretch	Add PII redaction rules before traces leave app.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 135 - Week 20: Tracing integration

Block	Instruction
Learn	Read only what is needed for this task. Topic: Tracing integration. Reference the primary docs for this phase before using courses.
Build	Instrument model calls, retrieval, tool gateway, and agent steps with trace IDs and spans.
Verify	One user request produces a full trace tree.
Stretch	Export traces to an observability backend.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 136 - Week 20: Prompt and model versioning

Block	Instruction
Learn	Read only what is needed for this task. Topic: Prompt and model versioning. Reference the primary docs for this phase before using courses.
Build	Track prompt templates, model versions, tool versions, and retrieval config per request.
Verify	A trace can tell exactly which config produced an answer.
Stretch	Add config diff view between two runs.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 137 - Week 20: Debugging failed runs

Block	Instruction
Learn	Read only what is needed for this task. Topic: Debugging failed runs. Reference the primary docs for this phase before using courses.
Build	Create a workflow for debugging: inspect trace, classify failure, reproduce with same inputs, patch, rerun eval.
Verify	Debug 5 real failures from your eval logs.
Stretch	Add run replay feature for saved traces.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 138 - Week 20: User feedback

Block	Instruction
Learn	Read only what is needed for this task. Topic: User feedback. Reference the primary docs for this phase before using courses.
Build	Capture thumbs, correction text, expected answer, and issue labels. Tie feedback to traces and eval datasets.
Verify	User feedback can create a new eval case.
Stretch	Add feedback triage dashboard.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 139 - Week 20: Operational metrics

Block	Instruction
Learn	Read only what is needed for this task. Topic: Operational metrics. Reference the primary docs for this phase before using courses.
Build	Create metrics: p50/p95 latency, cost per task, failure rate, refusal rate, tool failure rate, retrieval hit rate, approval rate.
Verify	Generate weekly ops report from your local data.
Stretch	Add alert thresholds for runaway cost or high failure rate.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 140 - Week 20: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Add a debugging playbook and trace screenshots to your docs.
Verify	Week 20 demo: full trace, replay, feedback-to-eval, ops metrics.
Stretch	Integrate Phoenix, LangSmith, or equivalent as an experiment.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 141 - Week 21: Eval strategy

Block	Instruction
Learn	Read only what is needed for this task. Topic: Eval strategy. Reference the primary docs for this phase before using courses.
Build	Define what quality means for each feature: exactness, groundedness, task success, safety, speed, cost, user satisfaction.
Verify	Create an eval plan for chat, RAG, and agents.
Stretch	Classify evals as unit, integration, offline, online.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 142 - Week 21: Golden datasets

Block	Instruction
Learn	Read only what is needed for this task. Topic: Golden datasets. Reference the primary docs for this phase before using courses.
Build	Build small high-quality datasets instead of giant noisy ones. Include edge cases and unanswerable cases.
Verify	Create 150 total eval cases across extraction, RAG, and agent tasks.
Stretch	Tag cases by difficulty and risk.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 143 - Week 21: Automated eval runner

Block	Instruction
Learn	Read only what is needed for this task. Topic: Automated eval runner. Reference the primary docs for this phase before using courses.
Build	Build a runner that executes cases against a chosen config and saves outputs, traces, scores, cost, and latency.
Verify	Run baseline config and save results.
Stretch	Add comparison report between two configs.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 144 - Week 21: LLM-as-judge carefully

Block	Instruction
Learn	Read only what is needed for this task. Topic: LLM-as-judge carefully. Reference the primary docs for this phase before using courses.
Build	Use judge models for rubric-based scoring, but calibrate with human review and examples.
Verify	Create rubrics for groundedness, helpfulness, citation quality, and safety.
Stretch	Measure judge disagreement on 20 cases.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 145 - Week 21: Agent trajectory evals

Block	Instruction
Learn	Read only what is needed for this task. Topic: Agent trajectory evals. Reference the primary docs for this phase before using courses.
Build	Evaluate not just final answer but steps: correct tool choice, safe args, no unnecessary calls, correct stopping.
Verify	Score 30 agent traces.
Stretch	Add failure labels: planning, tool selection, execution, synthesis, safety.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 146 - Week 21: CI regression gate

Block	Instruction
Learn	Read only what is needed for this task. Topic: CI regression gate. Reference the primary docs for this phase before using courses.
Build	Add a small smoke eval to CI and a larger nightly/local eval. Define pass thresholds.
Verify	A bad prompt change fails the eval gate.
Stretch	Add cost budget to eval runs.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 147 - Week 21: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Create an evaluation dashboard or report folder with trend charts.
Verify	Week 21 demo: eval harness, regression report, judge rubric, agent trajectory scoring.
Stretch	Add Ragas or another RAG evaluation tool as a comparative experiment.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 148 - Week 22: Routing principles

Block	Instruction
Learn	Read only what is needed for this task. Topic: Routing principles. Reference the primary docs for this phase before using courses.
Build	Define routing by task type, difficulty, data sensitivity, latency need, cost budget, and required reasoning depth.
Verify	Create routing matrix for extraction, chat, RAG, agent, and code tasks.
Stretch	Add a cheap/standard/deep mode flag.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 149 - Week 22: Implement model router

Block	Instruction
Learn	Read only what is needed for this task. Topic: Implement model router. Reference the primary docs for this phase before using courses.
Build	Build a router abstraction that supports provider, model, temperature, timeout, max tokens, fallback, and logging.
Verify	Endpoints call router instead of provider SDK directly.
Stretch	Add provider failover simulation.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 150 - Week 22: Cost controls

Block	Instruction
Learn	Read only what is needed for this task. Topic: Cost controls. Reference the primary docs for this phase before using courses.
Build	Add per-user and per-tenant daily budgets, max tokens, max steps, and tool-call limits.
Verify	Requests beyond budget degrade gracefully or require approval.
Stretch	Add budget dashboard.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 151 - Week 22: Latency controls

Block	Instruction
Learn	Read only what is needed for this task. Topic: Latency controls. Reference the primary docs for this phase before using courses.
Build	Measure p95. Add streaming, parallel retrieval, timeout budgets, early exits, smaller context, and cached summaries.
Verify	Reduce latency on one endpoint by at least 30 percent without major quality loss.
Stretch	Add latency breakdown by component.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 152 - Week 22: Caching and batching

Block	Instruction
Learn	Read only what is needed for this task. Topic: Caching and batching. Reference the primary docs for this phase before using courses.
Build	Cache safe repeated operations and batch embeddings/evals. Understand invalidation and privacy risks.
Verify	Embedding batch job is faster and cheaper than one-by-one calls.
Stretch	Add cache policy table to docs.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 153 - Week 22: Quality-cost experiments

Block	Instruction
Learn	Read only what is needed for this task. Topic: Quality-cost experiments. Reference the primary docs for this phase before using courses.
Build	Run evals across model choices and retrieval configs. Choose default based on quality per rupee/dollar and latency.
Verify	Produce a model/config comparison report.
Stretch	Add routing rule based on eval findings.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 154 - Week 22: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Document model routing architecture and operational tradeoffs.
Verify	Week 22 demo: router, budgets, caching, latency improvements, comparison report.
Stretch	Add LiteLLM or provider abstraction experiment if useful.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 155 - Week 23: Production architecture

Block	Instruction
Learn	Read only what is needed for this task. Topic: Production architecture. Reference the primary docs for this phase before using courses.
Build	Design services: UI, API, AI orchestrator, worker, vector DB, relational DB, cache, observability, tool gateway.
Verify	Create final production architecture diagram.
Stretch	Identify single points of failure.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 156 - Week 23: Docker and environments

Block	Instruction
Learn	Read only what is needed for this task. Topic: Docker and environments. Reference the primary docs for this phase before using courses.
Build	Containerize services with separate dev/staging/prod configs. Keep secrets out of images.
Verify	Full stack starts through Docker Compose.
Stretch	Add image build pipeline.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 157 - Week 23: Database and migrations

Block	Instruction
Learn	Read only what is needed for this task. Topic: Database and migrations. Reference the primary docs for this phase before using courses.
Build	Add migrations, seed data, backup/restore notes, and migration rollback plan.
Verify	Run migrations from scratch on a clean DB.
Stretch	Add test restore from backup.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 158 - Week 23: Security hardening

Block	Instruction
Learn	Read only what is needed for this task. Topic: Security hardening. Reference the primary docs for this phase before using courses.
Build	Review auth, RBAC, tenant isolation, secret management, CORS, SSRF, file upload safety, logging redaction, tool sandboxing.
Verify	Create a production readiness checklist and fix top issues.
Stretch	Run dependency audit and secret scan.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 159 - Week 23: SLOs and incident thinking

Block	Instruction
Learn	Read only what is needed for this task. Topic: SLOs and incident thinking. Reference the primary docs for this phase before using courses.
Build	Define SLOs for uptime, p95 latency, job completion, cost anomaly, and answer quality. Write incident playbooks.
Verify	Create 3 incident scenarios and response steps.
Stretch	Add alert rules for one scenario.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 160 - Week 23: Deploy

Block	Instruction
Learn	Read only what is needed for this task. Topic: Deploy. Reference the primary docs for this phase before using courses.
Build	Deploy to a platform you can operate. It can be simple, but it must include environment variables, DB, workers, logs, and a rollback path.
Verify	Public/private demo URL works and basic monitoring exists.
Stretch	Add blue/green or versioned deployment notes.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 161 - Week 23: Month 5 review

Block	Instruction
Learn	Read only what is needed for this task. Topic: Month 5 review. Reference the primary docs for this phase before using courses.
Build	Review production concepts. Your systems should now have queues, evals, tracing, routing, and deployment.
Verify	Month 5 checkpoint: production-shaped AI platform foundation.
Stretch	Write a readiness gap list before capstone month.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 162 - Week 24: Platform blueprint

Block	Instruction
Learn	Read only what is needed for this task. Topic: Platform blueprint. Reference the primary docs for this phase before using courses.
Build	Design an agent platform with agent definitions, tool registry, MCP adapters, policy engine, state store, memory, queue, tracing, evals, and UI.
Verify	Create a platform architecture spec.
Stretch	Add extensibility points for new agents and tools.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 163 - Week 24: Agent definition format

Block	Instruction
Learn	Read only what is needed for this task. Topic: Agent definition format. Reference the primary docs for this phase before using courses.
Build	Create a config format for agents: name, purpose, instructions, allowed tools, memory policy, model route, max budget, eval suite.
Verify	Load agent definitions from config and run them.
Stretch	Add validation for unsafe configs.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 164 - Week 24: Tool registry v2

Block	Instruction
Learn	Read only what is needed for this task. Topic: Tool registry v2. Reference the primary docs for this phase before using courses.
Build	Unify internal tools, MCP tools, and API tools behind one registry with schemas, permissions, timeouts, and versions.
Verify	One agent can use tools from multiple backends through same gateway.
Stretch	Add tool discovery UI.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 165 - Week 24: State and memory service

Block	Instruction
Learn	Read only what is needed for this task. Topic: State and memory service. Reference the primary docs for this phase before using courses.
Build	Separate workflow state, conversation memory, long-term memory, and audit logs. Define retention and deletion rules.
Verify	Implement state tables and memory APIs.
Stretch	Add memory eval cases to prevent bad memory writes.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 166 - Week 24: Policy and guardrails

Block	Instruction
Learn	Read only what is needed for this task. Topic: Policy and guardrails. Reference the primary docs for this phase before using courses.
Build	Apply policy before model call, before tool call, after tool result, before final answer, and before memory write.
Verify	Policy blocks at least 5 seeded unsafe scenarios.
Stretch	Add human override with audit.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 167 - Week 24: Developer experience

Block	Instruction
Learn	Read only what is needed for this task. Topic: Developer experience. Reference the primary docs for this phase before using courses.
Build	Create a local dev mode with mock models/tools, trace viewer, replay, eval runner, and sample agent templates.
Verify	A new agent can be added in under 30 minutes.
Stretch	Add CLI command to scaffold an agent.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 168 - Week 24: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Finalize capstone spec: what you will build, why it matters, success metrics, architecture, risks, and demo script.
Verify	Week 24 demo: agent platform skeleton and capstone plan.
Stretch	Ask for peer review from one strong engineer.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 169 - Week 25: Capstone slice 1: repo intelligence

Block	Instruction
Learn	Read only what is needed for this task. Topic: Capstone slice 1: repo intelligence. Reference the primary docs for this phase before using courses.
Build	Implement codebase ingestion, repo Q&A, architecture explanations, and cited file references.
Verify	User can ask how a feature works in a repo and receive grounded answer.
Stretch	Add PR/issue context if available.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 170 - Week 25: Capstone slice 2: task planning

Block	Instruction
Learn	Read only what is needed for this task. Topic: Capstone slice 2: task planning. Reference the primary docs for this phase before using courses.
Build	Agent converts a bug/feature request into an implementation plan with files, risks, tests, and approval gates.
Verify	Agent creates a clear plan for 5 sample engineering tasks.
Stretch	Add plan critique and revision.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 171 - Week 25: Capstone slice 3: safe coding workflow

Block	Instruction
Learn	Read only what is needed for this task. Topic: Capstone slice 3: safe coding workflow. Reference the primary docs for this phase before using courses.
Build	Read files, propose patch, wait for approval, apply patch, run whitelisted tests, summarize results.
Verify	Agent fixes one small bug in a sample repo with trace and rollback.
Stretch	Add multi-iteration debug loop with max steps.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 172 - Week 25: Capstone slice 4: MCP integration

Block	Instruction
Learn	Read only what is needed for this task. Topic: Capstone slice 4: MCP integration. Reference the primary docs for this phase before using courses.
Build	Expose selected capabilities through MCP and consume at least one MCP server through your gateway if safe.
Verify	MCP path works without bypassing policy/audit.
Stretch	Add MCP server trust configuration.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 173 - Week 25: Capstone slice 5: evals and traces

Block	Instruction
Learn	Read only what is needed for this task. Topic: Capstone slice 5: evals and traces. Reference the primary docs for this phase before using courses.
Build	Add eval suite for repo Q&A, planning, patch safety, and final answer quality. Add trace views.
Verify	Capstone has reproducible eval report and visible traces.
Stretch	Add comparison between two model routes.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 174 - Week 25: Capstone slice 6: production hardening

Block	Instruction
Learn	Read only what is needed for this task. Topic: Capstone slice 6: production hardening. Reference the primary docs for this phase before using courses.
Build	Review auth, tenant isolation, secrets, logging, queues, caching, cost controls, deployment, and incident docs.
Verify	Production readiness checklist is mostly green or has explicit gaps.
Stretch	Add smoke test after deployment.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 175 - Week 25: Weekly consolidation

Block	Instruction
Learn	Read only what is needed for this task. Topic: Weekly consolidation. Reference the primary docs for this phase before using courses.
Build	Create a demo script that shows the platform end to end in 10 minutes.
Verify	Week 25 demo: capstone works across core flows.
Stretch	Record a polished demo video.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 176 - Week 26: Bug bash

Block	Instruction
Learn	Read only what is needed for this task. Topic: Bug bash. Reference the primary docs for this phase before using courses.
Build	Use your evals, red-team set, and manual testing to identify the top 20 issues. Fix the top 10.
Verify	Bug list is prioritized by severity and user impact.
Stretch	Ask another dev to test it and file issues.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 177 - Week 26: Architecture case study

Block	Instruction
Learn	Read only what is needed for this task. Topic: Architecture case study. Reference the primary docs for this phase before using courses.
Build	Write a serious case study: problem, architecture, data flow, agent flow, security model, eval strategy, failures, tradeoffs.
Verify	Case study is good enough to send to a senior engineer.
Stretch	Turn diagrams into clean visuals.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 178 - Week 26: Portfolio packaging

Block	Instruction
Learn	Read only what is needed for this task. Topic: Portfolio packaging. Reference the primary docs for this phase before using courses.
Build	Clean README, setup guide, screenshots, demo video, eval report, threat model, and roadmap. Remove secrets and private data.
Verify	Repo is portfolio-ready or private-demo-ready.
Stretch	Create a landing page for the project.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 179 - Week 26: Level 3 gap review

Block	Instruction
Learn	Read only what is needed for this task. Topic: Level 3 gap review. Reference the primary docs for this phase before using courses.
Build	Assess gaps: distributed systems, deeper Python, Kubernetes, model serving, local inference, compilers/systems, security, eval science.
Verify	Create a personalized 6-month next roadmap.
Stretch	Pick one Level 3 specialization track.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 180 - Week 26: Interview and career narrative

Block	Instruction
Learn	Read only what is needed for this task. Topic: Interview and career narrative. Reference the primary docs for this phase before using courses.
Build	Prepare concise explanations for LLM apps, RAG, agents, MCP, evals, observability, security, and your capstone decisions.
Verify	Create a 20-question self-interview document.
Stretch	Practice explaining architecture without buzzwords.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 181 - Week 26: Final demo

Block	Instruction
Learn	Read only what is needed for this task. Topic: Final demo. Reference the primary docs for this phase before using courses.
Build	Run final demo end to end. Show problem -> agent plan -> retrieval -> tool calls -> approval -> patch/test -> trace -> eval report.
Verify	Final demo passes without manual database edits or hidden steps.
Stretch	Record final demo video and preserve trace IDs.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

Day 182 - Week 26: Final review and next commitment

Block	Instruction
Learn	Read only what is needed for this task. Topic: Final review and next commitment. Reference the primary docs for this phase before using courses.
Build	Review all notes and projects. Decide the next 6 months: agent platform depth, AI infra, applied product, or model systems.
Verify	Final checkpoint: you are Level 2 strong and have Level 3 foundations if you can build, secure, evaluate, and explain this platform.
Stretch	Publish or share the case study with selected engineers for feedback.
End note	Write 5 lines: what worked, what failed, what confused you, what you committed, and what must be improved later.

7. Project specifications

These projects are not optional. They are the proof that the curriculum converted into skill. Keep them in one monorepo if possible so later projects build on earlier infrastructure.

Project 1 - AI API Skeleton

A production-shaped AI backend with chat, streaming, structured outputs, tool calling, validation, logging, and tests.

Required capabilities	Must use provider SDK directly at least once Must expose typed endpoints Must track cost/latency Must include README and .env.example
Why it matters	This proves you understand LLM primitives without framework dependence.

Project 2 - Structured Extraction and Triage

Extract structured data from messy inputs and classify them into operational categories.

Required capabilities	Schema validation Batch eval dataset Retry only on safe errors Manual review threshold
Why it matters	This maps directly to business automation work.

Project 3 - Repo-Aware RAG Assistant

Index a MERN repo and answer questions with cited files and evidence.

Required capabilities	Code-aware chunking Hybrid retrieval Evaluation set No-answer behavior
Why it matters	This is the bridge between your current skill and AI-native engineering.

Project 4 - Agent Runtime From Scratch

Single-agent loop with tools, state, memory, step limits, and traces.

Required capabilities	Tool registry Max step/cost limits Trace viewer Approval for risky tools
Why it matters	This prevents framework-blindness.

Project 5 - LangGraph Workflow

Stateful workflow with branches, persistence, approval, and resume.

Required capabilities	Checkpoints Conditional routing Human approval Streaming progress
Why it matters	This teaches controllable orchestration.

Project 6 - MCP Tool Server

Expose safe capabilities to AI clients through MCP.

Required capabilities	Typed tools Resources Tests Client connection Security notes
Why it matters	This is the entry point to protocol-level AI tooling.

Project 7 - Secure Tool Gateway

Central gateway for validation, auth, policy, approval, execution, and audit.

Required capabilities	Policy checks Audit logs Approval tokens Sandboxed commands Tenant checks
Why it matters	This is Level 3-style infrastructure thinking.

Project 8 - Eval and Observability Platform

Trace AI workflows and run regression evals across models/prompts/retrieval configs.

Required capabilities	Trace tree Golden datasets Regression report Feedback-to-eval loop
Why it matters	This is what separates demos from production systems.

Project 9 - Capstone: AI Engineering Agent Platform

A deployed platform that combines repo RAG, planning, coding tools, MCP, policy, evals, traces, and deployment.

Required capabilities	End-to-end demo Threat model Eval report Architecture case study Production readiness checklist
Why it matters	This is your proof of relevance for the next stage.

8. Monthly assessment rubrics

Use these rubrics brutally. If a box is weak, fix it before pretending you finished the month. The market will not care that you watched content; it will care what you can build, explain, debug, and operate.

Month 1 Foundation Rubric

Pass condition
Can call LLM APIs directly from Node and Python.
Can use structured outputs and schema validation.
Can implement tool calling with safe execution boundaries.
Can stream responses and handle cancellation.
Can track latency, cost, errors, and prompt versions.

Month 2 RAG Rubric

Pass condition
Can design ingestion pipeline for docs, PDFs, web pages, and code.
Can explain chunking tradeoffs and retrieval failures.
Can implement hybrid search, metadata filters, and reranking.
Can evaluate retrieval separately from generation.
Can enforce permissions before retrieval context reaches the model.

Month 3 Agent Rubric

Pass condition
Can build an agent loop from scratch.
Can distinguish workflow from agent from multi-agent system.
Can implement LangGraph stateful workflows.
Can add memory, stop conditions, and human approval.
Can evaluate agent trajectories, not just final answers.

Month 4 MCP/Security Rubric

Pass condition
Can build MCP servers and expose tools/resources.
Can route MCP tool calls through a policy gateway.
Can threat model prompt injection, tool poisoning, and unsafe execution.
Can implement approvals, audit logs, sandboxing, and tenant checks.
Can explain why random MCP servers are supply-chain risk.

Month 5 Production Rubric

Pass condition
Can run long jobs through queues/workers.
Can instrument traces across model calls, retrieval, and tools.
Can build golden eval datasets and regression reports.
Can route models by cost, latency, and quality.

Pass condition
Can deploy and operate an AI backend with rollback and incident thinking.

Month 6 Capstone Rubric

Pass condition
Capstone combines RAG, agents, MCP, tool gateway, evals, traces, and deployment.
Architecture is documented clearly.
Security model is explicit and tested.
Eval report is reproducible.
Demo proves real engineering, not prompt-wrapper behavior.

9. Technical primers you should internalize

LLM application architecture

A production LLM feature is not just a prompt. It is an application pipeline. The minimum pipeline is request validation, context assembly, model call, output validation, persistence, observability, and fallback. Once tools are added, the pipeline expands to tool discovery, tool selection, input validation, permission checks, execution, result formatting, and audit. Once agents are added, the pipeline becomes a stateful loop with stopping conditions and safety limits.

- Model input is not only the user message. It includes instructions, retrieved context, memory, tool definitions, conversation history, and hidden runtime constraints.
- Model output is not only text. It may include structured JSON, tool calls, reasoning traces where exposed, refusal, or incomplete responses.
- Reliability comes from constraints around the model, not from asking the model to be reliable.
- The stronger your schemas, tests, evals, and policies, the safer your AI system becomes.

RAG architecture

RAG is a retrieval system plus a generation system. The model cannot cite evidence it never received. Most bad RAG demos fail because they treat vector search as magic. Strong RAG requires careful parsing, chunking, metadata, hybrid retrieval, reranking, permission filtering, evidence-aware prompting, and evaluation.

- Ingestion quality determines what can be retrieved later.
- Chunking should preserve meaning and references. For code, chunk by file/function when possible. For docs, chunk by headings. For PDFs, preserve page references.
- Hybrid retrieval matters for exact names, IDs, filenames, stack traces, and API signatures.
- Evaluation must separate retrieval quality from answer generation quality.
- Security requires permission filtering before context injection, not after the answer is generated.

Agent architecture

An agent is useful when the path to the answer or task completion is not fully known upfront. If the path is known, use a workflow. If the task needs external action, use tools. If the action can cause side effects, use approval and audit. If the task can run for a long time, use queues and checkpointed state.

- State must be explicit. Hidden state creates debugging nightmares.
- Stop conditions are mandatory: max steps, max time, max cost, max tool calls, max retries.
- Tool design is agent design. Bad tools produce bad agents.
- Memory should be deliberate. Do not let agents write permanent memory without policy.
- Multi-agent systems are useful only when role separation improves measurable quality.

MCP architecture

MCP is important because it standardizes how AI clients connect to external capabilities. For you, the value is not only using MCP servers. The value is knowing how to build them, secure them, route them through policy, and operate them as part of an AI platform.

- An MCP server should expose narrow tools and resources, not unlimited system access.
- Every MCP tool should have typed inputs, output limits, permission labels, and auditability.
- Untrusted MCP servers are supply-chain risk. Treat them like dependencies that can execute actions through your AI client.
- A serious architecture routes MCP tools through a tool gateway with policy, auth, approval, logging, and sandboxing.

Production AI system architecture

Production AI systems combine normal backend engineering with AI-specific concerns. You need queues for long jobs, traces for debugging, evals for quality, model routing for cost and latency, and security controls for tool use. This is where

senior engineering maturity matters.

- Use asynchronous workers for indexing, eval runs, report generation, and long-running agents.
- Use traces to connect model calls, retrieval, tool calls, workflow state, and final output.
- Use evals before changing prompts, chunking, retrieval configs, tool descriptions, or models.
- Use model routing to reserve expensive reasoning models for tasks that need them.
- Use policy gates before external actions, not after.

10. Cheat sheets

LLM Engineering Cheat Sheet

- Treat the model call as a remote probabilistic function with latency, cost, rate limits, and failure modes.
- Separate instructions from untrusted data. Label retrieved content as data, never as instructions.
- Use structured outputs for anything that feeds code, database writes, decisions, or downstream automations.
- Never trust model output without validation when the output controls software behavior.
- Track prompt version, model version, input size, output size, cost, latency, and trace ID for every serious call.
- Prefer smaller context with better retrieval over dumping everything into the prompt.

RAG Cheat Sheet

- RAG quality is usually retrieval quality, not model quality.
- Chunking should preserve semantic boundaries and metadata; bad chunks create bad answers.
- Use hybrid retrieval when exact names, IDs, APIs, filenames, or error strings matter.
- Evaluate retrieval separately from generation. If the right evidence is absent, the model cannot recover reliably.
- Every cited answer should map claims to source chunks. Refusal is a valid outcome.
- Permission filtering must happen before context reaches the model.

Agent Cheat Sheet

- An agent is a loop with state, tools, policy, and stop conditions. Autonomy without limits is a bug.
- Use workflows for known paths and agents for uncertain paths. Do not over-agent deterministic business logic.
- Tool design determines agent quality. Small, precise tools beat vague, powerful tools.
- Evaluate trajectories: plan, tool choice, arguments, observations, final response, and safety decisions.
- Human approval is required for writes, deletes, purchases, external messages, and code execution.
- Memory writes should be explicit, inspectable, and reversible.

MCP Cheat Sheet

- MCP standardizes how AI clients discover and call tools/resources from servers.
- Treat every MCP server as a supply-chain dependency, not a harmless plugin.
- Do not let MCP handlers bypass your auth, policy, approval, audit, and sandbox layers.
- Pin trusted servers, review tool definitions, and log tool version changes.
- Expose narrow tools with typed schemas and clear risk labels.
- Separate read-only capabilities from write or execution capabilities.

Production AI Systems Cheat Sheet

- Use queues for long-running tasks, indexing, evals, and agents. Keep request/response paths short.
- Use idempotency keys for retried jobs and tool actions.
- Model routing should consider task difficulty, cost, latency, risk, and data sensitivity.
- Observability must include prompts, retrieval, tool calls, model config, evals, and user feedback.
- Maintain golden datasets and regression gates before changing prompts, models, chunking, or retrieval.
- Cost is an architecture constraint, not an afterthought.

Security Cheat Sheet

- Assume user input, retrieved documents, web pages, tool output, and MCP servers are untrusted.

- Block prompt injection by separating instructions from data and by enforcing tool policy in code.
- Never execute raw commands generated by a model. Use allowlists, sandboxes, and approvals.
- Redact secrets from logs, traces, prompts, and tool outputs.
- Use least privilege for tools, databases, filesystems, APIs, and model-accessible context.
- Audit every external side effect with actor, tool, args hash, approval, result, and trace ID.

11. Templates

Daily learning log template

Field	What to write
Date / Day	Example: Day 43 - Hybrid retrieval.
Concept learned	One short paragraph in your own words.
Code shipped	Commit hash and what changed.
Failure observed	One bug, hallucination, bad retrieval, latency issue, or security issue.
Fix or next step	What you changed or what you will do tomorrow.
Confidence	0 to 5. 0 means confused; 5 means you can rebuild without notes.

Weekly demo template

Section	Content
Problem	What this week taught and why it matters.
Architecture	Diagram and explanation of main components.
Demo path	Exact steps to run and what to click/call.
Tests/evals	What proves it works. Include failures honestly.
Security	What can go wrong and what you did to reduce risk.
Next improvement	One concrete improvement for later.

Architecture decision record template

Field	Content
Decision	What did you decide?
Context	What problem or constraint forced the decision?
Options	What alternatives did you consider?
Tradeoffs	Cost, latency, complexity, security, maintainability.
Decision owner	You. Add date and version.
Review trigger	When should this decision be revisited?

AI feature threat model template

Field	Questions
Assets	What data, credentials, actions, or systems need protection?
Actors	User, admin, attacker, compromised tool, untrusted document, external API.
Trust boundaries	Where does untrusted input enter? Where do tools execute?
Attack paths	Prompt injection, tool poisoning, data exfiltration, unsafe command execution, cross-tenant leakage.
Controls	Validation, permission filtering, sandbox, approval, audit, redaction, rate limit.
Residual risk	What remains risky after controls?

12. After this curriculum: next 6 months toward Level 3

The six-month plan makes you strong at Level 2 and gives you real Level 3 foundations. To go deeper into Level 3, choose one specialization track instead of trying to master everything at once.

Track	Focus	Build next
Agent platform engineer	Runtime, state, tools, policies, evals, observability, developer experience.	Open-source-style agent runtime with plugin/tool gateway and eval harness.
AI infrastructure engineer	Queues, distributed systems, reliability, model routing, cost optimization, deployment.	Multi-tenant AI backend with autoscaling workers and operational dashboards.
AI security engineer	Prompt injection, MCP security, sandboxing, data exfiltration, supply chain, red teaming.	Agent red-team lab and secure tool execution platform.
RAG/search engineer	Hybrid search, reranking, knowledge graphs, permissions, freshness, evals.	Enterprise-grade knowledge assistant with advanced retrieval and source governance.
Model systems engineer	Local inference, quantization, model serving, GPUs, vLLM/Triton style stack.	Self-hosted inference service with routing and benchmarks.

Recommended next steps

- Deepen distributed systems fundamentals: queues, consistency, retries, idempotency, rate limiting, backpressure, observability.
- Study security seriously. Agent systems expand blast radius because models can select tools and actions.
- Build one public or private case study every quarter.
- Follow official docs and changelogs for OpenAI, Anthropic/MCP, LangGraph, Vercel AI SDK, LlamaIndex, Qdrant, and OWASP GenAI security.
- Do not chase every new framework. Learn primitives and evaluate frameworks by whether they improve control, safety, and operability.

13. Reference links

These references are intentionally concentrated around official documentation and durable engineering resources. Use them when the matching topic appears in the daily plan.

ID	Resource	URL
R1	OpenAI Platform Docs - Responses API, tools, structured outputs, embeddings, Agents SDK	https://developers.openai.com/api/docs
R2	OpenAI API Reference - Responses overview	https://developers.openai.com/api/reference/responses/overview
R3	OpenAI Structured Outputs guide	https://developers.openai.com/api/docs/guides/structured-outputs
R4	OpenAI Agents SDK docs - handoffs and orchestration	https://openai.github.io/openai-agents-python/
R5	Anthropic Model Context Protocol announcement	https://www.anthropic.com/news/model-context-protocol
R6	Model Context Protocol official docs	https://modelcontextprotocol.io/docs/getting-started/intro
R7	MCP Security Best Practices	https://modelcontextprotocol.io/docs/tutorials/security/security_best_practices
R8	LangGraph workflows and agents docs	https://docs.langchain.com/oss/python/langgraph/workflows-agents
R9	LangGraph GitHub repository	https://github.com/langchain-ai/langgraph
R10	Vercel AI SDK docs	https://ai-sdk.dev/docs/introduction
R11	LlamaIndex RAG documentation	https://developers.llamaindex.ai/python/framework/understanding/rag/
R12	Qdrant hybrid search docs	https://qdrant.tech/documentation/search/hybrid-queries/
R13	Ragas documentation	https://docs.ragas.io/en/latest/
R14	Arize Phoenix / OpenInference documentation	https://arize-ai.github.io/openinference/
R15	OWASP Top 10 for LLM Applications	https://owasp.org/www-project-top-10-for-large-language-model-applications/
R16	OWASP Top 10 for Agentic Applications 2026	https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/
R17	Full Stack Deep Learning	https://fullstackdeeplearning.com/
R18	DeepLearning.AI - AI Agents in LangGraph	https://learn.deeplearning.ai/courses/ai-agents-in-langgraph/information
R19	DeepLearning.AI - Building Agentic RAG with LlamaIndex	https://learn.deeplearning.ai/courses/building-agentic-rag-with-llamaindex/information
R20	Hugging Face Agents Course	https://huggingface.co/learn/agents-course/en/unit0/introduction

How to keep this curriculum current

- Re-check official docs monthly. AI APIs, MCP versions, SDKs, model names, and best practices change quickly.
- Treat provider SDK examples as volatile. The concepts last longer than exact APIs.
- Update your capstone when a major protocol, SDK, or security recommendation changes.
- Keep an internal changelog: what changed in your stack, why, and what broke.